

**INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DE SÃO PAULO
CAMPUS SÃO PAULO**

TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

**JOAO ANTÔNIO CHAMUSCA MARTINS
KELVYN GABRIEL MILU DE OLIVEIRA
LUCAS VIEIRA DE SOUZA
MATHEUS PEREIRA PEREIRA**

**SISTEMA INTELIGENTE DE MONITORAMENTO E SEGURANÇA
CIBERNÉTICA: APOIO À PROTEÇÃO E ORIENTAÇÃO SOBRE A
VULNERABILIDADE DIGITAL (SafeID)**

SÃO PAULO

2026

JOAO ANTÔNIO CHAMUSCA MARTINS
KELVYN GABRIEL MILU DE OLIVEIRA
LUCAS VIEIRA DE SOUZA
MATHEUS PEREIRA PEREIRA

**SISTEMA INTELIGENTE DE MONITORAMENTO E SEGURANÇA
CIBERNÉTICA: APOIO À PROTEÇÃO E ORIENTAÇÃO SOBRE A
VULNERABILIDADE DIGITAL (SafeID)**

Projeto apresentado ao Programa do Curso de Tecnologia em Análise e Desenvolvimento de Sistemas, do Instituto Federal de Educação, Ciência e Tecnologia de São Paulo, Campus de São Paulo, como requisito parcial para aprovação na disciplina SPOPIE1 – Projeto Integrado de Extensão 1, sob orientação do Professor Anderson Gomes Peixoto.

SÃO PAULO

2026

RESUMO

A crescente dependência de serviços digitais por indivíduos e organizações tem gerado um rastro massivo de informações pessoais e de acesso restrito, tornando-as vulneráveis a incidentes de segurança. No cenário atual, a exfiltração de dados em larga escala e a subsequente compilação de bases de credenciais expostas em repositórios de *threat intelligence* representam um risco crítico à privacidade e à integridade dos titulares. Diante dessa realidade, este trabalho apresenta o SafeID, um sistema desenvolvido para permitir ao usuário (cadastrado ou não) monitorar exposições registradas e oferecer apoio e orientação sobre sua vulnerabilidade digital.

O objetivo central do projeto é consolidar uma plataforma que atue no processamento e na triagem de informações provenientes de fontes externas que catalogam esses incidentes de segurança. Ao contrário de ferramentas de detecção em tempo real, o SafeID foca no mapeamento de dados que já foram comprometidos e estão circulando em repositórios de ameaças conhecidos. O sistema busca democratizar o acesso a essa informação técnica sobre o contexto da vulnerabilidade digital, permitindo que o cidadão comum compreenda se seus dados cadastrais ou credenciais de acesso fazem parte de vazamentos documentados, facilitando a gestão de sua própria identidade digital.

Metodologicamente, a pesquisa utiliza uma abordagem de desenvolvimento ágil para a criação de uma aplicação fundamentada em React (frontend) e Node.js com framework NestJS (backend). O diferencial do sistema reside na aplicação de modelos de inteligência artificial (LLMs) para realizar uma análise cognitiva dos incidentes encontrados, transformando registros técnicos em guias práticos de mitigação. Assim como, o SafeID propõe como entrega de valor um índice de risco visual — o "score de exposição" — e planos de ação personalizados. Dessa forma, a plataforma cumpre as diretrizes da Lei Geral de Proteção de Dados (LGPD) ao fortalecer a autodeterminação informativa e a cultura de segurança preventiva no Brasil.

Palavras-chaves: Segurança cibernética. Inteligência de Ameaças. LGPD. Inteligência Artificial.

ABSTRACT

The growing reliance on digital services by individuals and organizations has generated a massive trail of personal and restricted access information, making them vulnerable to security incidents. In the current scenario, large-scale data exfiltration and the subsequent compilation of exposed credential databases in threat intelligence repositories represent a critical risk to the privacy and integrity of data subjects. Given this reality, this paper presents SafeID, a system developed to allow users (registered or not) to monitor recorded exposures and offer support and guidance on their digital vulnerability.

The central objective of the project is to consolidate a platform that operates in the processing and screening of information from external sources that catalog these security incidents. Unlike real-time detection tools, SafeID focuses on mapping data that has already been compromised and is circulating in known threat repositories. The system seeks to democratize access to this technical information regarding the context of digital vulnerability, allowing ordinary citizens to understand if their registration data or access credentials are part of documented leaks, facilitating the management of their own digital identity.

Methodologically, the research uses an agile development approach to create an application based on React and Node.js with NestJS as back-end framework. The system's differential lies in the application of artificial intelligence models (LLMs) to perform a cognitive analysis of the incidents found, transforming technical records into practical mitigation guides. Furthermore, SafeID proposes as a value delivery a visual risk index — the "exposure score" — and personalized action plans. In this way, the platform complies with the guidelines of the General Data Protection Law (LGPD) by strengthening informative self-determination and the culture of preventive security in Brazil.

Keywords: Cybersecurity. Threat Intelligence. LGPD. Artificial Intelligence. SafeID.

ÍNDICE DE FIGURAS

Figura 1 - Diagrama de arquitetura, interface e componentes da aplicação	15
Figura 2 - Diagrama de arquitetura do SafeID	9
Figura 3 - Modelo Entidade Relacionamento (MER) SafeID	25
Figura 4 - Diagrama entidade-relacionamento SafeID	26
Figura 5 - Roadmap de desenvolvimento de atividades do SafeID	31

ÍNDICE DE TABELAS

Tabela 1 - Quadro Comparativo de Funcionalidades entre Concorrentes	15
Tabela 2 - Matriz de Responsabilidades e Atribuições da Equipe.....	19
Tabela 3 - Backlog do Produto (Product Backlog - PI-1 e PI-2)	22
Tabela 4 - Tabela de Requisitos Funcionais (RF).....	27
Tabela 5 - Tabela de Requisitos Não Funcionais (RNF)	29
Tabela 6 - Dicionário de dados do SafeID	27
Tabela 7 - Cronograma Consolidado e Macro-Fases de Engenharia do Projeto	30
Tabela 8 - Tabela de custos operacionais mensais (OpEx) estimados por categoria	32

ÍNDICE DE ABREVIACÕES E SIGLAS

ACID – Atomicidade, Consistência, Isolamento e Durabilidade
AES – Advanced Encryption Standard (Padrão de Criptografia Avançada)
API – Application Programming Interface (Interface de Programação de Aplicações)
AWS – Amazon Web Services
B2C – Business-to-Consumer
B2B2C – Business-to-Business-to-Consumer
CI/CD – Continuous Integration / Continuous Deployment (Integração Contínua / Implantação Contínua)
CNPJ – Cadastro Nacional da Pessoa Jurídica
CPF – Cadastro de Pessoas Físicas
CPU – Central Processing Unit (Unidade Central de Processamento)
CRUD – Create, Read, Update, Delete (Criar, Ler, Atualizar, Deletar)
CSS – Cascading Style Sheets (Folhas de Estilo em Cascata)
DBA – Database Administrator (Administrador de Banco de Dados)
DDL – Data Definition Language (Linguagem de Definição de Dados)
DER – Diagrama Entidade-Relacionamento
DML – Data Manipulation Language (Linguagem de Manipulação de Dados)
ECS – Elastic Container Service
GCM – Galois/Counter Mode
HIBP – Have I Been Pwned
IA – Inteligência Artificial
IFSP – Instituto Federal de Educação, Ciência e Tecnologia de São Paulo
JSON – JavaScript Object Notation (Notação de Objetos JavaScript)
JWT – JSON Web Token
LGPD – Lei Geral de Proteção de Dados Pessoais
LLM – Large Language Model (Modelo de Linguagem de Grande Escala)
MFA – Multi-Factor Authentication (Autenticação em Múltiplos Fatores)
MVP – Minimum Viable Product (Produto Mínimo Viável)
NLP – Natural Language Processing (Processamento de Linguagem Natural)
ORM – Object-Relational Mapping (Mapeamento Objeto-Relacional)
OSINT – Open Source Intelligence (Inteligência de Fontes Abertas)
PII – Personally Identifiable Information (Informações Pessoais Identificáveis)
PO – Product Owner (Dono do Produto)
PoC – Proof of Concept (Prova de Conceito)
RAM – Random Access Memory (Memória de Acesso Aleatório)
RDS – Relational Database Service
SBDG – Sistema de Gerenciamento de Banco de Dados
SHA – Secure Hash Algorithm
UDF – User Defined Functions (Funções Definidas pelo Usuário)
UML – Unified Modeling Language (Linguagem de Modelagem Unificada)
UX – User Experience (Experiência do Usuário)
VPC – Virtual Private Cloud

SUMÁRIO

1. INTRODUÇÃO.....	9
1.1 Objetivos.....	11
1.1.1 Objetivo Geral	11
1.1.2 Objetivo Específico	11
1.2. Problema e Solução Proposta	12
1.2.1 Problema.....	12
1.2.2 Solução Proposta.	12
1.3 Justificativa.....	13
1.4 Análise da Concorrência.....	14
1.4.1 Have I Been Pwned (HIBP).....	14
1.4.2 Mozilla Monitor.....	14
1.4.3 Soluções de Identidade (Ex: LifeLock, Aura)	14
1.5 Quadro Comparativo de Funcionalidades	15
1.6 Layout, Arquitetura e Interface	15
2 REVISÃO DA LITERATURA	16
2.1 A Economia da Informação e a Evolução das Ameaças Digitais.....	16
2.2 Custódia de Dados, Fadiga de Segurança e o "Gap" de Detecção	17
2.3 Marco Regulatório (LGPD) e a Literacia Digital no Brasil	17
3 GESTÃO DO PROJETO	18
3.1 Organização da Equipe.....	18
3.1.1 Responsabilidades e Atividades	18
3.2 Metodologias de Gestão e Desenvolvimento	20
3.2.1 Scrum (Metodologia Ágil).....	20
3.3 Repositório da Aplicação.....	20
3.3.1 Definição do Repositório.....	21
3.3.2 Link do Repositório e Especificações para Acesso	21
3.3.3 Backlog do Produto (Product Backlog).....	21
3.3.4 Detalhamento das Sprints (Projeto Integrado I - Março a Junho)	23
3.3.5 Planejamento para Projeto Integrado II (Agosto a Dezembro)	24
4 DESENVOLVIMENTO DO PROJETO	25
4.1 Escopo do projeto	25
4.1.1 Regras de negócio.....	26
4.1.2 Requisitos funcionais.....	27
4.1.3 Requisitos não funcionais.....	29
4.2 Histórias de usuário	31
4.2.1 Descrição das Histórias de Usuário	31
4.3 Arquitetura.....	34

4.3.1 Definições da arquitetura.....	34
4.3.2 Diagrama da arquitetura	9
4.4 Tecnologias.....	9
4.4.1 Front-end: React.js e Tailwind CSS	9
4.4.2 Back-end: Node.js, NestJS e Prisma ORM	10
4.4.3 Banco de Dados: PostgreSQL	11
4.4.4 Inteligência Artificial e APIs Externas	12
4.4.5 Infraestrutura em Cloud.....	13
4.5 Testes e Manutenibilidade	15
4.5.1 Plano de Testes	15
4.5.2 Análise Estática	16
4.5.3 Testes Funcionais	16
4.5.4 Testes Não Funcionais.....	18
4.5.5 Testes Automatizados.....	20
4.5.6 Logs	20
4.5.7 Code Convention	21
4.6 Segurança, Privacidade e Legislação.....	22
4.6.1 Critérios de Segurança e Privacidade	22
4.6.2 Observância à Legislação	23
4.7 Modelo de Banco de Dados.....	24
4.7.1 Modelo Entidade-Relacionamento (MER).....	24
4.7.2 Diagrama Entidade-Relacionamento (DER)	25
4.7.3 Dicionário de Dados	27
4.8 Duração / Cronograma.....	28
4.8.1 Análise da Duração do Projeto	29
5 VIABILIDADE FINANCEIRA	31
5.1 Custos	31
5.2 Receitas.....	32
5.3 Cenário Realista.....	33
5.4 Cenário Otimista.....	33
5.5 Cenário Pessimista.....	34
6 CONSIDERAÇÕES FINAIS	35
6.1 Dificuldades, Escolhas e Descartes	35
REFERÊNCIAS BIBLIOGRÁFICAS	38

1. INTRODUÇÃO

A transformação digital observada nas últimas décadas tem promovido mudanças significativas na forma como indivíduos, organizações e governos produzem, armazenam e compartilham informações. Com a ampliação do acesso à internet e a intensificação do uso de serviços digitais, dados pessoais passaram a ser constantemente coletados e processados em diferentes plataformas, tornando-se ativos de grande valor econômico e estratégico. Nesse cenário, a segurança da informação assume papel fundamental, especialmente no que se refere à proteção da privacidade e à integridade dos dados dos usuários (VON SOLMS et al., 2013).

Paralelamente a esse avanço tecnológico, observa-se o crescimento expressivo de incidentes relacionados ao vazamento de dados. Esses eventos, caracterizados pela exposição indevida de informações sensíveis, têm se tornado cada vez mais frequentes e complexos, afetando milhões de usuários em escala global. Segundo estudos recentes, a exploração de vulnerabilidades em sistemas, aliada a práticas inadequadas de gestão da informação, contribui significativamente para o aumento desses incidentes (ZHANG et al., 2022). Além disso, a sofisticação das ameaças cibernéticas, como ataques de engenharia social e campanhas automatizadas de coleta de credenciais, intensifica o cenário de risco.

No contexto brasileiro, a problemática da exposição de dados pessoais apresenta características particulares. O aumento do uso de plataformas digitais, impulsionado por fatores como inclusão digital e expansão de serviços online, tem ampliado a quantidade de informações sensíveis disponíveis na rede. Entretanto, esse crescimento não foi acompanhado, na mesma proporção, por uma cultura consolidada de segurança digital entre os usuários. Pesquisas apontam que práticas como reutilização de senhas, ausência de autenticação em múltiplos fatores e desconhecimento sobre vazamentos são recorrentes, tornando os indivíduos mais vulneráveis a fraudes e ataques (NEMEC ZLATOLAS et al., 2024).

Outro aspecto relevante refere-se às consequências desses vazamentos. Para além da exposição de dados, tais incidentes podem resultar em prejuízos financeiros, danos à reputação e comprometimento da privacidade dos indivíduos. No Brasil, estudos recentes indicam que o impacto econômico associado a violações de dados tem apresentado crescimento contínuo, refletindo não apenas falhas técnicas, mas também lacunas em políticas de proteção e conscientização dos usuários (CHEN; HENRY; JIANG, 2023). Dessa forma, o problema ultrapassa a dimensão tecnológica, configurando-se também como uma questão social e educacional.

Apesar da existência de ferramentas voltadas à identificação de vazamentos de dados, observa-se que grande parte dessas soluções apresenta limitações relevantes. Em geral, tais sistemas fornecem informações de forma técnica, sem considerar o nível de compreensão do usuário comum. Como consequência, mesmo quando o indivíduo tem acesso à informação sobre a exposição de seus dados, ele pode não ser capaz de interpretar corretamente o risco ou adotar medidas adequadas de mitigação. Nesse sentido, a literatura destaca a importância de abordagens que integrem aspectos tecnológicos e educacionais, facilitando a compreensão e promovendo o uso seguro de sistemas digitais (CHEN; WANG, 2024).

Diante desse cenário, torna-se evidente a necessidade de soluções que não apenas identifiquem a exposição de dados, mas também auxiliem o usuário na interpretação das informações e na tomada de decisões. O uso de tecnologias baseadas em inteligência artificial apresenta-se como uma alternativa promissora nesse contexto, uma vez que permite a análise de grandes volumes de dados e a geração de respostas adaptadas ao perfil do usuário. Estudos recentes indicam que sistemas inteligentes podem contribuir significativamente para a comunicação de riscos de forma mais acessível e personalizada (REUBEN-OWOH, 2025).

Com base nessas premissas, o presente trabalho propõe o desenvolvimento do **SafeID**, uma plataforma web voltada ao monitoramento da exposição digital de usuários comuns. A solução integra mecanismos de verificação de vazamentos por meio de APIs especializadas em monitoramento de incidentes e inteligência de ameaças, um modelo de cálculo de risco baseado em regras e um assistente inteligente capaz de interpretar os resultados e fornecer orientações em linguagem acessível. Diferentemente das abordagens tradicionais, o SafeID busca atuar não apenas como uma ferramenta de diagnóstico, mas também como um instrumento de conscientização e apoio à tomada de decisão.

A proposta fundamenta-se na ideia de que a proteção de dados pessoais deve ser acessível a todos os usuários, independentemente de seu nível de conhecimento técnico. Ao traduzir informações complexas em recomendações claras e práticas, o sistema contribui para a redução de riscos associados a fraudes, invasões de contas e outras ameaças digitais. Dessa forma, o SafeID alinha-se às demandas contemporâneas por soluções que conciliem tecnologia, usabilidade e educação digital.

Por fim, destaca-se que a relevância deste estudo está diretamente relacionada à crescente dependência de sistemas digitais e à necessidade de fortalecimento da cultura de segurança da informação. Ao propor uma abordagem centrada no usuário e apoiada por tecnologias emergentes, este trabalho busca contribuir para o desenvolvimento de soluções

mais eficazes no enfrentamento dos desafios relacionados à exposição de dados na sociedade digital.

1.1 Objetivos

1.1.1 Objetivo Geral

Desenvolver a plataforma **SafeID**, um sistema inteligente de apoio à decisão focado em segurança cibernética preventiva, capaz de processar informações provenientes de repositórios de *threat intelligence* focados em incidentes de exposição de dados para diagnosticar a exposição de dados pessoais e fornecer diretrizes de mitigação personalizadas por meio de inteligência artificial.

1.1.2 Objetivo Específico

- **Investigar, selecionar e consumir** APIs de monitoramento de exposições de incidentes de segurança *Open-Source Intelligence* (OSINT) que permitam a triagem automatizada de credenciais e dados cadastrais expostos;
- **Modelar e implementar** um algoritmo de cálculo de *Risk Score* (índice de risco) que pondere a criticidade do vazamento com base na natureza dos dados comprometidos reportados pelas APIs;
- **Desenvolver** uma interface web responsiva que utilize elementos de visualização de dados (como o termômetro de risco) para facilitar a interpretação dos usuários;
- **Projetar uma arquitetura de persistência** que registre o histórico de consultas do usuário, armazenando os metadados dos incidentes em estruturas de dados flexíveis para acompanhamento evolutivo da segurança;
- **Integrar** um modelo de linguagem de grande escala (LLM) para atuar como assistente cognitivo, traduzindo metadados técnicos em planos de ação práticos e acessíveis;
- **Aplicar** os princípios de Privacy by Design e as diretrizes da Lei Geral de Proteção de Dados (LGPD), assegurando o tratamento ético e a proteção das informações coletadas durante o ciclo de uso do sistema.

1.2. Problema e Solução Proposta

1.2.1 Problema

A aceleração da transformação digital consolidou os dados pessoais como ativos de alto valor, porém, essa transição resultou em um rastro massivo de informações sensíveis vulneráveis. O cenário atual é marcado pela escala industrial dos incidentes: somente em 2023, o Brasil foi alvo de mais de **32,8 bilhões de tentativas de ataques cibernéticos**, consolidando-se como o segundo país mais atingido na América Latina (FORTINET, 2024). Somado a isso, o custo médio global de um vazamento de dados atingiu a marca histórica de **US\$ 4,45 milhões**, representando um aumento de 15% em relação aos três anos anteriores (IBM SECURITY, 2023).

Apesar da magnitude desses indicadores, o titular dos dados enfrenta uma grave **assimetria de informação**. Enquanto grupos cibercriminosos compilam e comercializam bases de dados exfiltradas em repositórios de *threat intelligence* (onde bilhões de credenciais são disponibilizadas anualmente), o usuário comum raramente possui consciência da extensão de sua própria vulnerabilidade. Ferramentas de consulta existentes frequentemente entregam resultados em formatos técnicos brutos que não permitem uma interpretação imediata do risco (CONTI; DARGAHI; DEGHANTANHA, 2018). Essa lacuna gera inércia, pois o indivíduo, mesmo ciente do vazamento, não compreende a gravidade da exposição ou quais contramedidas devem ser priorizadas.

1.2.2 Solução Proposta.

Diante dessa lacuna, propõe-se o desenvolvimento do **SafeID**, uma plataforma inteligente de apoio à decisão que atua na camada de pós-incidente. A solução baseia-se no processamento de metadados provenientes de APIs especializadas em monitoramento de exposições (como *Have I Been Pwned*). O diferencial do SafeID reside na integração de três componentes centrais:

1. **Mapeamento Automatizado:** A triagem de incidentes vinculados ao identificador do usuário em repositórios de ameaças;
2. **Cálculo de Risk Score:** Um algoritmo ponderado que quantifica a gravidade da exposição com base na natureza do dado (ex: senhas possuem peso superior a nomes);
3. **Assistente Cognitivo:** O uso de modelos de linguagem (LLMs) para converter diagnósticos técnicos em planos de ação personalizados e compreensíveis.

Dessa forma, a plataforma transforma dados brutos de incidentes em literacia em segurança, permitindo que o usuário adote contramedidas eficazes para a proteção de sua identidade digital.

1.3 Justificativa

A relevância deste projeto fundamenta-se na necessidade de transpor as barreiras técnicas que impedem o cidadão comum de exercer sua **autodeterminação informativa**, direito assegurado pela Lei Geral de Proteção de Dados (LGPD). No Brasil, o crescimento exponencial de vazamentos não foi acompanhado por uma educação digital equivalente, tornando o fator humano o elo mais vulnerável na cadeia de segurança (NEMEC ZLATOLAS et al., 2024).

O desenvolvimento do SafeID é viabilizado e orientado através de uma parceria estratégica com a **Enetsec** (<https://enetsec.com.br/>), empresa especializada em consultoria de segurança cibernética e inteligência em tecnologia da informação. A colaboração com a Enetsec confere ao projeto uma dimensão de aplicação real, permitindo que os requisitos técnicos e as funcionalidades de análise de risco sejam validados por profissionais atuantes no mercado de segurança. Esta sinergia entre a academia e o setor privado assegura que o SafeID não apenas cumpra os requisitos pedagógicos da disciplina, mas também atenda às demandas latentes do mercado B2B2C, servindo como uma solução eficaz para a proteção da identidade digital de seus usuários e colaboradores sob as diretrizes da LGPD.

A justificativa para o desenvolvimento do SafeID sustenta-se em pilares estratégicos:

- **Mitigação de Riscos Sociais:** A ferramenta combate a vulnerabilidade cognitiva ao oferecer orientação clara em um cenário onde o desconhecimento sobre vazamentos é facilitador de fraudes financeiras e roubo de identidade.
- **Inovação Tecnológica:** O projeto explora a convergência de tecnologias emergentes, como a integração de APIs de segurança e o processamento de linguagem natural (NLP), para resolver um problema crítico de usabilidade e comunicação de risco cibernético (REUBEN-OWOH; HAIG, 2025).
- **Conformidade Legal:** Ao facilitar o diagnóstico de exposições históricas (CHEN; HENRY; JIANG, 2023), o sistema auxilia na redução de danos à reputação e prejuízos financeiros, fortalecendo a cultura de segurança preventiva no país.

1.4 Análise da Concorrência

O mercado de soluções voltadas à verificação de exposição de dados apresenta ferramentas consolidadas, variando de agregadores de incidentes a suítes completas de proteção de identidade. Contudo, a análise técnica revela que a maioria foca na **entrega do dado bruto**, negligenciando a camada de **interpretação e suporte à decisão** para o usuário leigo.

1.4.1 *Have I Been Pwned (HIBP)*

Referência global em agregação de incidentes, o HIBP destaca-se pela robustez de sua base de dados OSINT. No entanto, sua interface é estritamente técnica e informativa.

- **Limitação:** O sistema opera no modelo "consulta e resposta", sem fornecer uma métrica de gravidade (Score) ou um plano de mitigação que vá além de sugestões genéricas como a troca de senhas. É a fonte primária de dados do SafeID, mas não um assistente de segurança.

1.4.2 *Mozilla Monitor*

O serviço da Mozilla atua como uma camada de interface sobre os dados do HIBP, buscando humanizar a experiência do usuário.

- **Limitação:** Embora possua uma interface mais amigável e ofereça alertas por e-mail, as recomendações permanecem estáticas. Não há processamento de linguagem natural ou análise cognitiva dos vazamentos para gerar orientações específicas para diferentes perfis de incidentes.

1.4.3 *Soluções de Identidade (Ex: LifeLock, Aura)*

Plataformas comerciais de alto custo que oferecem monitoramento extensivo (incluindo *credit score* e vigilância em tempo real).

- **Limitação:** Além da barreira financeira, essas soluções apresentam alta complexidade de configuração e são frequentemente integradas a serviços bancários e de crédito, o que as afasta do objetivo de uma ferramenta ágil e focada exclusivamente na **literacia em cibersegurança do cidadão comum**.

1.5 Quadro Comparativo de Funcionalidades

Abaixo, apresenta-se a comparação entre os concorrentes analisados e a proposta do SafeID, evidenciando o diferencial competitivo no uso de IA e Score de Risco.

Tabela 1 - Quadro Comparativo de Funcionalidades entre Concorrentes

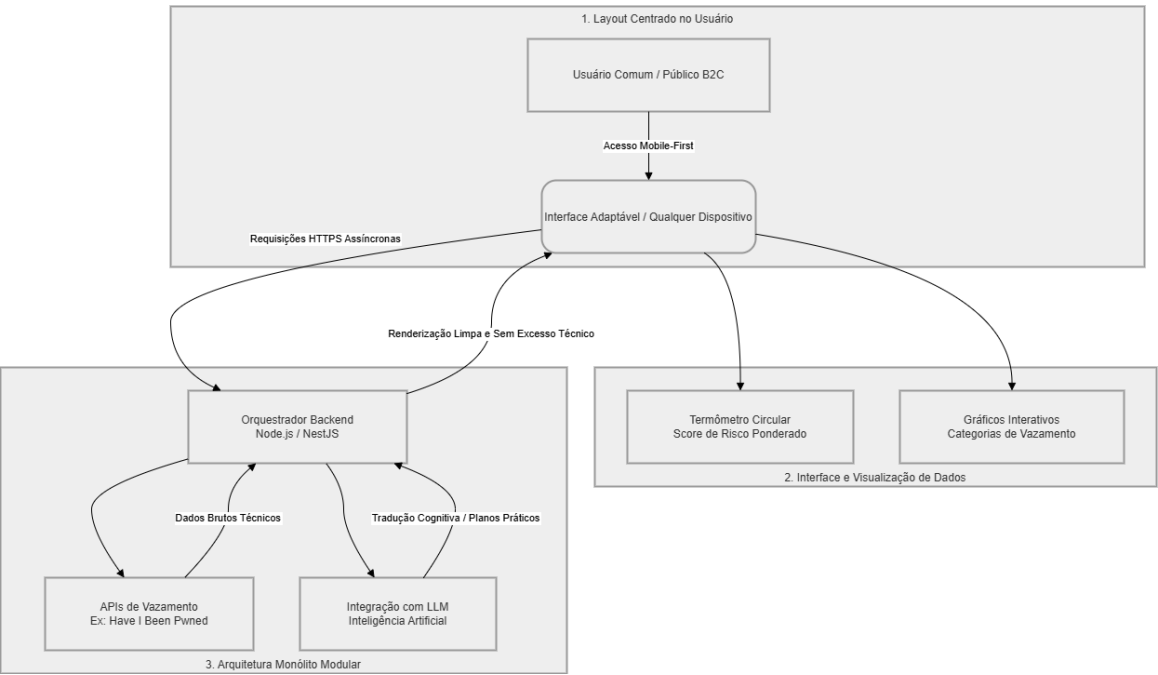
Funcionalidade	HIBP	Mozilla Monitor	Soluções Pagas	SafeID
Consulta de incidentes (OSINT)	Sim	Sim	Sim	Sim
Alertas de novas exposições	Não*	Sim	Sim	Sim
Interface Acessível	Não	Sim	Parcial	Sim
Score de Risco Ponderado	Não	Não	Sim	Sim
Assistente Cognitivo (LLM)	Não	Não	Não	Sim
Foco em Educação Digital	Não	Parcial	Não	Sim

*A funcionalidade gratuita de consulta do HIBP não inclui alertas ativos para o usuário comum sem registro de domínio.

1.6 Layout, Arquitetura e Interface

Para que o SafeID se diferencie dos concorrentes, sua arquitetura foi projetada para ser leve e centrada no usuário, conforme o diagrama de blocos abaixo:

Figura 1 - Diagrama de arquitetura, interface e componentes da aplicação



1. **Layout:** Utiliza o padrão *Mobile First* para garantir que o usuário possa consultar sua segurança em qualquer dispositivo.
2. **Arquitetura:** Construída sob o padrão de Monólito Modular com processamento assíncrono. Para garantir que o consumo de dados da LLM (Inteligência Artificial) não comprometa a performance da API principal e a experiência do usuário, as rotinas de alta latência são delegadas a filas de mensageria em segundo plano.
3. **Interface:** Foca na visualização de dados (gráficos para categorias de vazamento e um termômetro circular para o Score de Risco), eliminando o excesso de textos técnicos presentes nos concorrentes.

2 REVISÃO DA LITERATURA

2.1 A Economia da Informação e a Evolução das Ameaças Digitais

A transição para a economia digital reconfigurou o dado pessoal de um simples registro cadastral para um ativo estratégico de alto valor estratégico. De acordo com **Von Solms e Van Niekerk (2013)**, a cibersegurança evoluiu de uma disciplina estritamente técnica e organizacional para uma dimensão de governança pessoal, onde a proteção da identidade digital do indivíduo torna-se o núcleo da defesa em rede. Esta mudança de paradigma reflete a onipresença dos sistemas de informação na vida cotidiana, exigindo que a segurança acompanhe a mobilidade e a dispersão dos dados.

Contudo, essa valorização dos dados gerou o que a literatura define como "Economia da Exposição". Dados exfiltrados de grandes plataformas não são incidentes isolados, mas alimentam um mercado secundário de inteligência cibernética. **Conti, Dargahi e Dehghantanha (2018)** argumentam que a análise de ameaças baseada em evidências (*Threat Intelligence*) é a única forma de antecipar riscos em um ecossistema onde bilhões de credenciais já foram comprometidas e estão disponíveis para exploração.

O cenário brasileiro é um reflexo crítico dessa vulnerabilidade global. Relatórios recentes da **Fortinet (2024)** indicam que o Brasil sofreu mais de 32,8 bilhões de tentativas de ataques cibernéticos em um único ano, consolidando-se como o segundo país mais visado da América Latina. Esse volume massivo de tentativas de invasão justifica a necessidade de ferramentas que atuem no monitoramento constante de exposições, uma vez que a superfície de ataque se expandiu proporcionalmente à digitalização dos serviços financeiros e sociais no país.

2.2 Custódia de Dados, Fadiga de Segurança e o "Gap" de Detecção

A robustez da segurança individual é frequentemente insuficiente devido às falhas na custódia de dados por grandes serviços digitais. Um indicador alarmante apresentado pela **IBM Security (2023)** revela que o ciclo de vida médio de um vazamento de dados, desde a sua ocorrência até a efetiva contenção, é de aproximadamente 277 dias. Este intervalo temporal é crítico, pois permite que informações sensíveis circulem livremente em repositórios de acesso restrito muito antes de o titular ser notificado ou adotar medidas de proteção.

Além do atraso na detecção, a literatura aponta para o fenômeno da "fadiga de segurança" como um agravante dos riscos. **Nemec Zlatolas et al. (2024)** discutem que a complexidade na gestão de múltiplas credenciais e a subutilização de métodos de Autenticação em Múltiplos Fatores (MFA) são consequências diretas da falta de ferramentas que traduzam o risco técnico para o usuário leigo. O problema reside na entrega de dados brutos sem o devido contexto, o que impede uma reação assertiva por parte do cidadão comum.

Dessa forma, a eficácia das contramedidas depende diretamente da capacidade de interpretação do usuário sobre o incidente. **Santos e Pinto (2025)** reforçam que a percepção de risco é distorcida por barreiras cognitivas e linguísticas, sugerindo que sistemas de segurança devem evoluir para interfaces de apoio à decisão. O SafeID busca preencher justamente esta lacuna, transformando metadados de incidentes em orientações práticas, reduzindo o hiato de resposta entre a descoberta do vazamento e a ação mitigadora.

2.3 Marco Regulatório (LGPD) e a Literacia Digital no Brasil

A consolidação da Lei Geral de Proteção de Dados (LGPD - Lei 13.709/2018) estabeleceu no Brasil o direito fundamental à transparência e à autodeterminação informativa. No entanto, a aplicação prática desses preceitos esbarra na baixa literacia digital da população brasileira. Conforme analisado por **Chen, Henry e Jiang (2023)**, a simples divulgação de fatores de risco cibernético não é informativa o suficiente se não houver uma estrutura que permita ao titular dos dados compreender o impacto real daquela exposição em sua privacidade.

A necessidade de mediação tecnológica torna-se evidente quando observamos a disparidade entre o rigor normativo e a execução técnica. **Reuben-Owoh e Haig (2025)** destacam que a adoção de padrões voluntários de cibersegurança deve ser acompanhada por mecanismos que facilitem a comunicação de riscos de forma personalizada. No contexto brasileiro, onde a dependência de plataformas digitais é elevada, a proteção de dados deve ser

encarada não apenas como conformidade jurídica, mas como um processo de educação e empoderamento do usuário.

Portanto, o SafeID integra-se a este cenário como um instrumento de apoio à conformidade e cidadania digital. Ao alinhar as diretrizes da LGPD com tecnologias de inteligência artificial, o sistema permite que o usuário exerça seu direito de monitoramento de forma autônoma. Conforme concluem **Chen e Wang (2024)**, a gestão social de incidentes cibernéticos exige abordagens que integrem a resposta técnica ao suporte educacional, promovendo uma cultura de segurança preventiva que transcenda o ambiente corporativo e alcance o cidadão comum.

3 GESTÃO DO PROJETO

3.1 Organização da Equipe

A equipe do SafeID é composta por quatro membros, distribuídos em funções que cobrem o ciclo completo de desenvolvimento, desde a concepção do produto até a implementação e persistência dos dados.

- **Matheus Pereira Pereira:** Product Owner e Scrum Master. Responsável pela visão do produto, gestão do *Backlog* e facilitação dos ritos ágeis.
- **Lucas Vieira de Souza:** Desenvolvedor Back-end. Responsável pela lógica de servidor, integração com APIs externas e inteligência artificial.
- **Joao Antônio Chamusca Martins:** Desenvolvedor Front-end. Responsável pela interface do usuário, responsividade e experiência do usuário (UX).
- **Kelvyn Gabriel Milu de Oliveira:** Desenvolvedor e Administrador de Banco de Dados (DBA). Responsável pela modelagem, integridade e performance das estruturas de dados.

3.1.1 Responsabilidades e Atividades

As atividades foram distribuídas de forma colaborativa, utilizando o modelo de matriz de responsabilidades. O PO/Scrum Master atua na interface com os requisitos de negócio, enquanto o time de desenvolvimento (DevTeam) foca na implementação técnica, garantindo que as integrações entre o Front-end, Back-end e Banco de Dados sejam coesas.

Tabela 2 - Matriz de Responsabilidades e Atribuições da Equipe

Integrante	Papel no Scrum	Especialidade Técnica	Atribuições e Entregas
Matheus Pereira	Scrum Master / PO	Gestão e Dados	Liderança da documentação acadêmica, gestão do <i>Backlog</i> , curadoria dos dados das APIs e validação do cálculo de risco (<i>Score</i>).
Kelvyn Gabriel	Desenvolvedor / DBA	Cibersegurança	Modelagem do banco de dados (PostgreSQL), segurança da arquitetura, definição dos pesos de criticidade dos vazamentos e conformidade com a LGPD.
Lucas Vieira	Desenvolvedor	Back-end / APIs	Construção da API do SafeID (Node.js), integração com o <i>Have I Been Pwned</i> , conexão com a LLM (Inteligência Artificial) e lógica de processamento de metadados.
João Antônio	Desenvolvedor	Front-end / UX	Design da interface (Figma), desenvolvimento dos componentes em React, implementação do termômetro de risco e garantia da responsividade.

3.1.1.1 Sinergia entre Artefatos e Documentação

A gestão do projeto foi desenhada para que a construção conceitual e a técnica caminhem juntas:

- **Documentação Coletiva:** Embora o Matheus lidere a redação acadêmica, o Kelvyn e o Lucas e João são responsáveis por documentar as especificações técnicas (Dicionário de Dados, Diagrama de Classes e Documentação da API no GitHub).
- **Repositório Único (Monorepo):** Optamos por um repositório centralizado no GitHub. Isso facilita o rastreamento de quem desenvolveu cada *feature* e garante que a documentação técnica esteja sempre próxima ao código (*README.md* e *Wiki*).

- **Integração de Conhecimentos:** A experiência profissional do Kelvyn em cibersegurança alimenta diretamente a lógica do Lucas nas APIs, enquanto o Matheus traduz essa complexidade técnica para a linguagem acessível da documentação e da interface projetada pelo João.

3.2 Metodologias de Gestão e Desenvolvimento

O projeto adota o **Scrum** como metodologia ágil de gestão, permitindo entregas incrementais e adaptação rápida a mudanças de requisitos durante o processo de construção.

3.2.1 Scrum (Metodologia Ágil)

O Scrum foi selecionado por sua eficácia em projetos de tecnologia onde a integração de diferentes camadas (IA, APIs e Banco de Dados) exige ciclos constantes de inspeção e adaptação.

3.2.1.1 Sprints

As atividades foram organizadas em ciclos quinzenais (*Sprints*), iniciados em março de 2024.

- **Planejamento:** No início de cada Sprint, a equipe define os itens do *Backlog* a serem desenvolvidos.
- **Construção e Documentação:** Paralelamente ao código, a documentação técnica é atualizada para garantir o registro da arquitetura.
- **Reuniões de Alinhamento:** Realizadas para identificar impedimentos técnicos, especialmente na integração com as APIs de monitoramento de incidentes.

3.3 Repositório da Aplicação

A gestão de configuração e versão do código-fonte é realizada por meio de ferramentas de controle de versão distribuídas.

3.3.1 Definição do Repositório

Foi adotado o **Git** como sistema de controle de versão, utilizando o **GitHub** como plataforma de hospedagem. Esta escolha justifica-se pela facilidade de colaboração entre a equipe e a integração contínua (CI/CD) com ferramentas de deploy.

3.3.2 Link do Repositório e Especificações para Acesso

O projeto **SafeID** adota o modelo de **Monorepo**, centralizando toda a inteligência de dados, documentação e código-fonte em um único repositório gerenciado dentro de uma organização no GitHub. Essa estrutura garante a rastreabilidade das entregas e a integridade entre o desenvolvimento técnico e a documentação acadêmica.

Organização dos Diretórios:

- **/client**: Contém o código-fonte da interface web desenvolvida em **React.js**.
- **/server**: Contém a lógica de negócio, integração com APIs OSINT e o motor de IA em **Node.js**.
- **/database**: Scripts de migração (DDL/DML), modelagem Entidade-Relacionamento e exemplos de esquemas de dados.
- **/docs**: Documentação oficial do projeto (incluindo o arquivo Word e versões em PDF), além de registros de requisitos.
- **/assets**: Diagramas de arquitetura, logotipos e recursos visuais do sistema.

Link de Acesso: <https://github.com/Safe-ID/safeid-main>

O repositório está configurado com um arquivo **README.md** na raiz que orienta a navegação e o processo de instalação. As entregas de cada sprint são evidenciadas através do histórico de *commits* e *logs* de atividades dos membros, permitindo a auditoria das contribuições individuais mencionadas no cronograma e validando a evolução do Produto Mínimo Viável (MVP).

A aplicação SafeID encontra-se disponível e operacional no endereço: <https://www.safeid.app.br>, podendo ser acessada publicamente para validação das funcionalidades implementadas.

3.3.3 Backlog do Produto (Product Backlog)

Seguindo as boas práticas no desenvolvimento de sistemas com a metodologia empregada, o Backlog do SafeID foi categorizado para permitir uma visão clara do que compõe o Protótipo (PI-1) e a Versão Final (PI-2).

Tabela 3 - Backlog do Produto (Product Backlog - PI-1 e PI-2)

ID	Categoria	Descrição do Item	Fase	Prioridade
01	Requisito	Levantamento de requisitos funcionais e não funcionais.	PI-1	Essencial
02	Requisito	Pesquisa e seleção de APIs de monitoramento (OSINT).	PI-1	Essencial
03	Arquitetura	Definição da stack tecnológica (React, Node.js, DB).	PI-1	Essencial
04	Banco de Dados	de Modelagem do Diagrama Entidade-Relacionamento (DER).	PI-1	Essencial
05	Banco de Dados	de Implementação do esquema de tabelas para histórico e usuários.	PI-1	Alta
06	Front-end	Prototipação de alta fidelidade (Figma).	PI-1	Alta
07	Back-end	Desenvolvimento do servidor API (Node.js + NestJS)	PI-1	Essencial
08	Integração	Implementação do módulo de consumo da API externa (HIBP).	PI-1	Essencial
09	Lógica	Desenvolvimento do algoritmo de cálculo de <i>Risk Score</i> .	PI-1	Essencial
10	Front-end	Implementação da tela de consulta por identificador (E-mail).	PI-1	Alta
11	Front-end	Desenvolvimento do Dashboard de visualização de vazamentos.	PI-1	Alta
12	Documentação	Redação dos Capítulos 3 e 4 (Gestão e Análise).	PI-1	Essencial
13	Segurança	Implementação de autenticação e autorização (JWT/Login).	PI-2	Alta
14	Inteligência	Integração com LLM para interpretação de dados técnicos.	PI-2	Alta
15	Inteligência	Geração automática de plano de ação preventivo (via IA).	PI-2	Alta

16	Back-end	Módulo de armazenamento de logs de consulta e auditoria.	PI-2	Média
17	Segurança	Criptografia de dados sensíveis em repouso (LGPD).	PI-2	Essencial
18	Front-end	Implementação de temas (Dark/Light mode) e acessibilidade.	PI-2	Baixa
19	Notificação	Sistema de envio de alertas automáticos via e-mail.	PI-2	Média
20	Qualidade	Realização de testes de integração e carga.	PI-2	Alta
21	Segurança	Testes de penetração (Pentest) e correção de vulnerabilidades.	PI-2	Essencial
22	Documentação	Finalização da monografia e revisão para banca final.	PI-2	Essencial

3.3.4 Detalhamento das Sprints (Projeto Integrado I - Março a Junho)

O cronograma de PI-1 foca na fundamentação teórica e na construção do protótipo funcional (MVP), cobrindo os itens **01 a 12** do Product Backlog.

Sprint 1: Concepção e Engenharia de Requisitos (Março)

- **Objetivo:** Estabelecer a base conceitual e técnica para o monitoramento de vazamentos.
- **Itens do Backlog:** 01, 02 e 12.
- **Artefatos:** Documento de Visão, Levantamento de Requisitos e Estudo de APIs (OSINT).
- **Entregas:** Definição do Escopo, Referencial Teórico e Configuração do Repositório.

Sprint 2: Modelagem e Design de Experiência (Abril)

- **Objetivo:** Traduzir os requisitos em arquitetura de dados e interface visual.
- **Itens do Backlog:** 03, 04, 05 e 06.
- **Artefatos:** DER (Diagrama Entidade-Relacionamento), Dicionário de Dados e Protótipo de Alta Fidelidade (Figma).
- **Entregas:** Arquitetura do Banco de Dados e Layout da Interface Homem-Máquina.

Sprint 3: Desenvolvimento do Core e Lógica de Risco (Maio)

- **Objetivo:** Implementar o motor de busca e o algoritmo de análise.
- **Itens do Backlog:** 07, 08 e 09.

- **Artefatos:** Servidor NestJS funcional, Módulo de integração HIBP e Algoritmo de cálculo de *Risk Score*.
- **Entregas:** Backend operacional capaz de processar e categorizar incidentes reais.

Sprint 4: Integração Front-end e Entrega do Protótipo (Junho)

- **Objetivo:** Consolidar a Prova de Conceito (PoC) para apresentação à banca.
- **Itens do Backlog:** 10, 11 e 12.
- **Artefatos:** Dashboard em React integrado ao Backend e Relatório Técnico Parcial.
- **Entregas:** Protótipo funcional com consulta e visualização de risco; Documentação de PI-1 finalizada.

3.3.5 Planejamento para Projeto Integrado II (Agosto a Dezembro)

Para o segundo semestre, o Roadmap foca na evolução do protótipo para um sistema robusto, implementando inteligência avançada e segurança rigorosa (Itens **13 a 22**).

Agosto e Setembro: Autenticação e Inteligência Cognitiva

- **Foco:** Garantir a privacidade do usuário e a camada de IA.
- **Itens do Backlog:** 13, 14, 15 e 16.
- **Atividades:** Implementação de autenticação JWT, integração com a API da LLM para interpretação dos vazamentos e geração do plano de ação preventivo personalizado.

Outubro: Conformidade, Segurança e Notificações

- **Foco:** Alinhamento com a LGPD e monitoramento ativo.
- **Itens do Backlog:** 17, 18 e 19.
- **Atividades:** Implementação de criptografia em repouso (banco de dados), desenvolvimento do sistema de alertas via e-mail e ajustes de acessibilidade e interface (Dark/Light mode).

Novembro: Garantia de Qualidade e Resiliência

- **Foco:** Estabilização do sistema para ambiente de produção.
- **Itens do Backlog:** 20 e 21.
- **Atividades:** Execução de testes de carga, testes de penetração (Pentest) para identificar vulnerabilidades e correções finais na lógica do sistema.

Dezembro: Encerramento e Defesa

- **Foco:** Consolidação dos resultados e apresentação final.
- **Item do Backlog:** 22.

- **Atividades:** Revisão ortográfica e técnica da monografia completa, preparação dos slides de defesa e entrega final do sistema SafeID funcional à banca examinadora.

4 DESENVOLVIMENTO DO PROJETO

4.1 Escopo do projeto

O escopo do projeto **SafeID** compreende o desenvolvimento de uma aplicação web voltada à literacia e proteção de dados pessoais, atuando especificamente na camada de pós-incidente de segurança. A solução propõe-se a servir como uma ponte inteligível entre bases de dados técnicas de vazamentos (*data breaches*) e o usuário final, utilizando inteligência artificial para personalizar orientações de mitigação.

O que será desenvolvido (Inclusões):

- **Módulo de Consulta:** Interface para que o usuário submeta identificadores (como e-mail) para verificação em bases de dados OSINT (Open Source Intelligence), utilizando integração via API com provedores consolidados no mercado de cibersegurança.
- **Motor de Análise de Risco:** Algoritmo que processa a natureza do vazamento (ex: exposição de senhas, endereços IP, nomes completos) e atribui uma métrica de gravidade ponderada (*Risk Score*).
- **Assistente de Orientação (IA):** Integração com modelos de linguagem de larga escala (LLM) para traduzir metadados técnicos em planos de ação preventivos e compreensíveis para o público leigo.
- **Painel de Controle (Dashboard):** Visualização histórica das consultas realizadas e evolução do nível de exposição do usuário.

O que não será desenvolvido (Exclusões):

- **Recuperação de Contas:** O sistema não realizará ações diretas em contas de terceiros (ex: o SafeID não trocará a senha do e-mail do usuário automaticamente).
- **Monitoramento de Rede Local:** A aplicação não atuará como antivírus ou firewall residente no dispositivo do usuário.
- **Venda de Proteção Financeira:** O projeto não oferece seguros ou garantias financeiras contra fraudes, limitando-se à orientação e monitoramento informativo.

Premissas e Restrições da aplicação

- **Premissa:** O sistema depende da disponibilidade das APIs de terceiros (como *Have I Been Pwned*). Caso o serviço externo esteja offline, o SafeID apresentará uma

mensagem de indisponibilidade momentânea, não sendo responsabilidade do grupo a manutenção de bases de dados de terceiros.

- **Restrição:** O sistema não realizará "quebra de hashes" ou descriptografia de senhas. Se o vazamento indicar que a senha está criptografada, o sistema apenas informará o status, cumprindo seu papel informativo e ético.

O projeto será entregue em duas etapas: no primeiro semestre, um **Protótipo Funcional (PoC)** validando a comunicação com APIs e a lógica de risco; no segundo semestre, a **Aplicação Integral** com interface finalizada, sistema de autenticação e o assistente cognitivo plenamente integrado.

4.1.1 Regras de negócio

As regras de negócio a seguir definem as restrições e o funcionamento lógico do SafeID, assegurando que o sistema opere em conformidade com as diretrizes de cibersegurança e proteção de dados.

- **RN01 – Identificador Único de Consulta:** O sistema deve aceitar como identificador primário para consulta o endereço de e-mail. Para a fase de PI-2, o sistema poderá ser expandido para outros identificadores, desde que validados por máscaras específicas.
- **RN02 – Ponderação do Score de Risco:** O cálculo do *Risk Score* deve ser baseado na criticidade dos dados expostos em cada vazamento. Dados como "Senhas" e "Dados Financeiros" possuem peso maior (ex: peso 10) do que "Nomes de Usuário" ou "Endereços IP" (ex: peso 2). O score final é a média ponderada de todas as ocorrências encontradas.
- **RN03 – Temporalidade do Risco:** Vazamentos ocorridos nos últimos 24 meses devem ter um impacto maior no score de risco do que vazamentos mais antigos, considerando que a probabilidade de a credencial ainda estar ativa é superior em incidentes recentes.
- **RN04 – Limitação de Consultas (Rate Limiting):** Para evitar abusos e garantir a estabilidade da integração com APIs externas, cada usuário (ou IP não autenticado no protótipo) terá um limite de 3 consultas por hora.
- **RN05 – Anonimização de Resultados Sensíveis:** O sistema nunca deverá exibir a senha vazada em texto claro. Caso o metadado do vazamento inclua a senha, o SafeID deve apenas informar a existência da exposição e orientar a troca imediata, preservando a segurança do usuário mesmo dentro da plataforma.

- **RN06 – Personalização via IA:** O plano de ação gerado pela Inteligência Artificial deve ser obrigatoriamente contextualizado ao tipo de vazamento encontrado. Se o vazamento for de "Dados de Cartão", a recomendação prioritária deve ser o bloqueio bancário; se for de "Senha", a troca de credenciais e ativação de MFA.
- **RN07 – Histórico de Privacidade:** Consultas realizadas por usuários não autenticados (visitantes) não serão armazenadas no banco de dados após o encerramento da sessão, visando o princípio de minimização da LGPD. Somente usuários cadastrados terão seus históricos preservados para fins de acompanhamento evolutivo.

4.1.2 Requisitos funcionais

Os requisitos funcionais descrevem as funções que o software deve executar para atender às necessidades dos usuários e as regras de negócio estabelecidas. Para o SafeID, os requisitos foram divididos entre o que será entregue no protótipo (PI-1) e a implementação final (PI-2).

Tabela 4 - Tabela de Requisitos Funcionais (RF)

ID	Requisito Funcional	Descrição	Fase
RF01	Consulta por Identificador (OSINT)	O sistema deve permitir a entrada de um endereço de e-mail e orquestrar, em <i>background</i> via filas distribuídas (BullMQ), a consulta à API <i>Have I Been Pwned</i> (HIBP), garantindo o respeito ao limite de requisições externas (<i>Rate Limit</i>).	PI-1
RF02	Categorização de Incidentes	O sistema deve processar o <i>payload</i> de retorno da API, extraindo e categorizando metadados estruturados, como nome da plataforma comprometida e tipologia dos dados vazados (ex: senhas em texto claro, IPs, hashes).	PI-1
RF03	Exibição do <i>Risk Score</i>	O sistema deve processar os dados expostos através do Motor de Risco interno, atribuindo pesos matemáticos de criticidade e gerando uma pontuação final (<i>Risk Score</i>), a ser exibida graficamente no <i>dashboard</i> do usuário.	PI-1

RF04	Tratamento de Erros e Validação	O sistema deve validar a entrada através de esquemas rígidos (Class-validator (NestJS)) e utilizar o padrão de resiliência (<i>Fail Fast / Circuit Breaker</i>) para informar ao usuário sobre e-mails inválidos, ausência de vazamentos ou indisponibilidade temporária das APIs.	PI-1
RF05	Geração de Dashboard Resumido	O front-end em React deve renderizar uma visualização consolidada, traduzindo as métricas brutas de risco e os incidentes técnicos em uma interface intuitiva e "Mobile First".	PI-1
RF06	Geração de Recomendações de Segurança Mitigatórias	O sistema deve processar os dados das chaves expostas e, por meio de integração com os modelos de linguagem natural da API OpenAI/Azure (6.41.0) , gerar um conjunto de recomendações personalizadas para o utilizador.	PI-2
RF07	Emissão de Diagnóstico de Risco Cognitivo	A aplicação deve utilizar inteligência artificial generativa (API OpenAI/Azure (6.41.0)) para traduzir dados técnicos de vazamentos massivos em relatórios simples, legíveis e didáticos para o utilizador final de perfil B2C.	PI-2
RF08	Gestão de Contas de Usuário	O sistema deve prover um módulo de autenticação, permitindo o armazenamento seguro (via PostgreSQL/Prisma) do histórico de consultas vinculadas ao perfil do usuário, aplicando criptografia em repouso.	PI-2
RF09	Monitoramento Ativo de Ameaças	O ecossistema deve orquestrar <i>Background Jobs</i> recorrentes para consultar silenciosamente as bases de dados públicas, disparando alertas proativos ao usuário cadastrado em caso de novas exposições detectadas.	PI-2
RF10	Exportação de Relatório (PDF)	O sistema deve permitir a consolidação do <i>Dashboard</i> e dos planos de ação gerados pela IA	PI-2

em um documento portátil e padronizado (PDF),
viabilizando a consulta *offline*.

4.1.3 Requisitos não funcionais

Enquanto os funcionais dizem *o que* o sistema faz, os não funcionais descrevem *como* ele deve ser (qualidade, segurança e performance).

Tabela 5 - Tabela de Requisitos Não Funcionais (RNF)

ID	Requisito Não Funcional	Categoria	Descrição
RNF01	Privacidade por Design (LGPD)	Segurança	O sistema não deve trafegar ou armazenar e-mails consultados em texto claro para serviços de cache, utilizando técnicas de ofuscação (Hash SHA-256) no Redis e criptografia simétrica em repouso feita na aplicação, campo String (AES-256-GCM) no banco PostgreSQL.
RNF02	Desempenho e Orquestração	Desempenho	O sistema deve otimizar o tempo de resposta priorizando o cache de memória (Redis). Para consultas novas, deve respeitar os limites rígidos de requisições da API externa (HIBP) utilizando filas distribuídas (BullMQ).
RNF03	Integração com o Motor Cognitivo	Usabilidade	O ecossistema de inteligência artificial deve ser obrigatoriamente intermediado pelo SDK oficial OpenAI/Azure (6.41.0), garantindo que o tempo de resposta percebido pelo utilizador, desde a solicitação até

			a entrega do diagnóstico cognitivo, não ultrapasse 3 segundos.
RNF04	Resiliência e Tolerância a Falhas	Confiabilidade	A aplicação deve implementar o padrão <i>Circuit Breaker</i> (Opossum) para cortar a comunicação com APIs externas caso a taxa de erro ultrapasse 50%, prevenindo lentidão em cascata e exaustão de recursos do servidor.
RNF05	Compatibilidade e Responsividade	Portabilidade	A interface front-end deve adotar a abordagem "Mobile First", sendo totalmente responsiva e compatível com navegadores desktop e dispositivos móveis atuais.
RNF06	Segurança de Tráfego e API	Segurança	Além de toda comunicação trafegar via HTTPS/TLS, a API deve aplicar proteção contra abusos limitando as requisições por IP (<i>Rate Limiting</i>) e ocultar assinaturas tecnológicas nos cabeçalhos HTTP utilizando middlewares de <i>Hardening</i> (Helmet.js).
RNF07	Integridade e Sanitização de Dados	Segurança	Toda entrada de dados recebida pela API deve passar por um funil rigoroso de validação e tipagem (utilizando esquemas Class-validator (NestJS)) para garantir o formato RFC de e-mails e prevenir vulnerabilidades como <i>Buffer Overflow</i> e injeções.
RNF08	Manutenibilidade e Qualidade (DevOps)	Manutenibilidade	O código-fonte deve manter uma cobertura de testes de unidade (Jest) superior a 80%, sendo validado automaticamente por esteiras de CI/CD (GitHub Actions) e

empacotado para produção utilizando contêineres (*Docker Multi-stage Build*).

4.2 Histórias de usuário

4.2.1 Descrição das Histórias de Usuário

US01 – Consulta de Vulnerabilidade (PI-1)

- História: Como cidadão preocupado com minha segurança, eu quero inserir meu e-mail principal no sistema para verificar se minhas informações foram expostas em vazamentos de dados conhecidos.
- Critérios de Aceite:
 - O sistema deve validar o formato do e-mail inserido utilizando esquemas de validação estrita (ex: Class-validator (NestJS)).
 - A requisição deve ser enfileirada (via BullMQ) para respeitar os limites de taxa da API externa, exibindo um *feedback* visual (loading) ao usuário.
 - Em caso de indisponibilidade da API externa, o sistema deve acionar o *Circuit Breaker* e exibir uma mensagem amigável de erro.

US02 – Entendimento do Nível de Risco (PI-1)

- História: Como usuário leigo em tecnologia, eu quero visualizar um índice de risco (score) de fácil compreensão para que eu saiba a urgência de tomar medidas de proteção.
- Critérios de Aceite:
 - O score deve ser apresentado de forma visual (ex: termômetro ou cores de semáforo).
 - O valor numérico deve ser processado pelo Motor de Risco interno, classificando os dados expostos em níveis de criticidade (1 a 4).
 - Deve haver uma breve legenda explicando o que o nível "Baixo", "Médio" ou "Alto" significa.

US03 – Tradução de Metadados via IA (PI-2)

- História: Como usuário sem conhecimento técnico, eu quero que a inteligência artificial explique em linguagem simples o que significa cada termo técnico do vazamento (ex:

"SQL Injection" ou "Combolist") para que eu entenda o impacto real na minha privacidade.

- Critérios de Aceite:
 - O sistema deve enviar à LLM (OpenAI/Azure (6.41.0)) apenas os metadados do vazamento, sem enviar o e-mail real do usuário, preservando a privacidade.
 - O texto deve evitar jargões complexos, focando na consequência prática para o usuário.

US04 – Plano de Ação Preventivo (PI-2)

- História: Como vítima de uma exposição de dados, eu quero receber orientações passo a passo do que fazer para mitigar o dano, para que eu possa proteger minhas contas de forma assertiva.
- Critérios de Aceite:
 - As recomendações devem ser dinâmicas e baseadas na resposta da IA sobre os dados vazados.
 - O plano deve incluir links ou tutoriais simples sobre como ativar a autenticação em duas etapas (MFA).

US05 – Monitoramento de Histórico (PI-2)

- História: Como usuário cadastrado, eu quero salvar meu histórico de consultas para que eu possa acompanhar se as medidas que tomei reduziram meu score de vulnerabilidade ao longo do tempo.
- Critérios de Aceite:
 - O sistema deve exigir login seguro para acessar o histórico.
 - O histórico deve ser recuperado do banco de dados (PostgreSQL) descriptografando os registros salvos com AES-256-GCM.
 - O Dashboard deve exibir um gráfico comparativo entre a primeira consulta e a situação atual.

US06 – Autenticação e Privacidade (PI-2)

- História: Como usuário recorrente, eu quero criar uma conta protegida para que minhas consultas e dados de histórico não fiquem expostos a terceiros que utilizem o mesmo dispositivo.
- Critérios de Aceite:
 - O sistema deve permitir o cadastro e validar o e-mail em conformidade com as normas de higienização de dados.
 - As senhas devem ser salvas utilizando *hash* irreversível.

- O acesso deve ser gerido via tokens seguros (JWT), sendo interrompido após inatividade.

US07 – Gestão de Múltiplos Identificadores (PI-2)

- História: Como responsável pela segurança digital da minha família, eu quero cadastrar múltiplos e-mails no meu painel para monitorar a vulnerabilidade de todos os meus dependentes em um único lugar.
- Critérios de Aceite:
 - O usuário deve conseguir adicionar e remover e-mails da sua lista de monitoramento.
 - O dashboard deve separar os resultados por identificador.

US08 – Notificações de Alerta Ativo (PI-2)

- História: Como usuário preventivo, eu quero ser notificado via e-mail assim que um novo vazamento contendo meus dados for detectado, para que eu possa agir antes que ocorra uma invasão real.
- Critérios de Aceite:
 - O sistema deve orquestrar *Background Jobs* para realizar varreduras automáticas periódicas silenciosas.
 - O alerta deve conter um link direto para o plano de ação sugerido.

US09 – Exportação de Plano de Mitigação (PI-2)

- História: Como usuário que deseja se organizar offline, eu quero exportar o plano de ação gerado pela IA em formato PDF para que eu possa segui-lo como um checklist de segurança.
- Critérios de Aceite:
 - O documento gerado deve conter o resumo do vazamento e as instruções passo a passo.
 - O arquivo deve ser leve e formatado adequadamente para leitura.

US10 – Administração e Auditoria (PI-2)

- História: Como administrador do sistema, eu quero visualizar métricas agregadas (anonimizadas) de consultas para entender quais são os vazamentos mais comuns e treinar melhor o modelo de IA.
- Critérios de Aceite:
 - O painel administrativo deve exibir o volume de consultas por período.

- O sistema deve aplicar ofuscação (ex: Hash SHA-256) em informações analíticas, garantindo que nenhum e-mail identificável seja exposto nesta visão (Privacy by Design).

4.3 Arquitetura

4.3.1 Definições da arquitetura

O SafeID adota uma arquitetura de **Monólito Modular** (*Modular Monolith*). Diferentemente de uma arquitetura de microsserviços puros — que introduziria complexidade desnecessária de orquestração e rede (*over-engineering*) para o escopo deste MVP —, o monólito modular mantém a aplicação em uma base de código unificada através do *framework* NestJS, mas com limites de domínio estritamente separados. O alto desempenho e o isolamento de falhas não ocorrem pela fragmentação física da aplicação, mas sim pelo processamento assíncrono: tarefas de alta latência e operações *I/O-bound*, como a consulta à API OSINT externa e a inferência de Inteligência Artificial, são desacopladas da *thread* principal e enviadas para *workers* de processamento via filas (Redis e BullMQ). Essa abordagem entrega a robustez de processamento necessária, mantendo a simplicidade operacional e de implantação de um monólito.

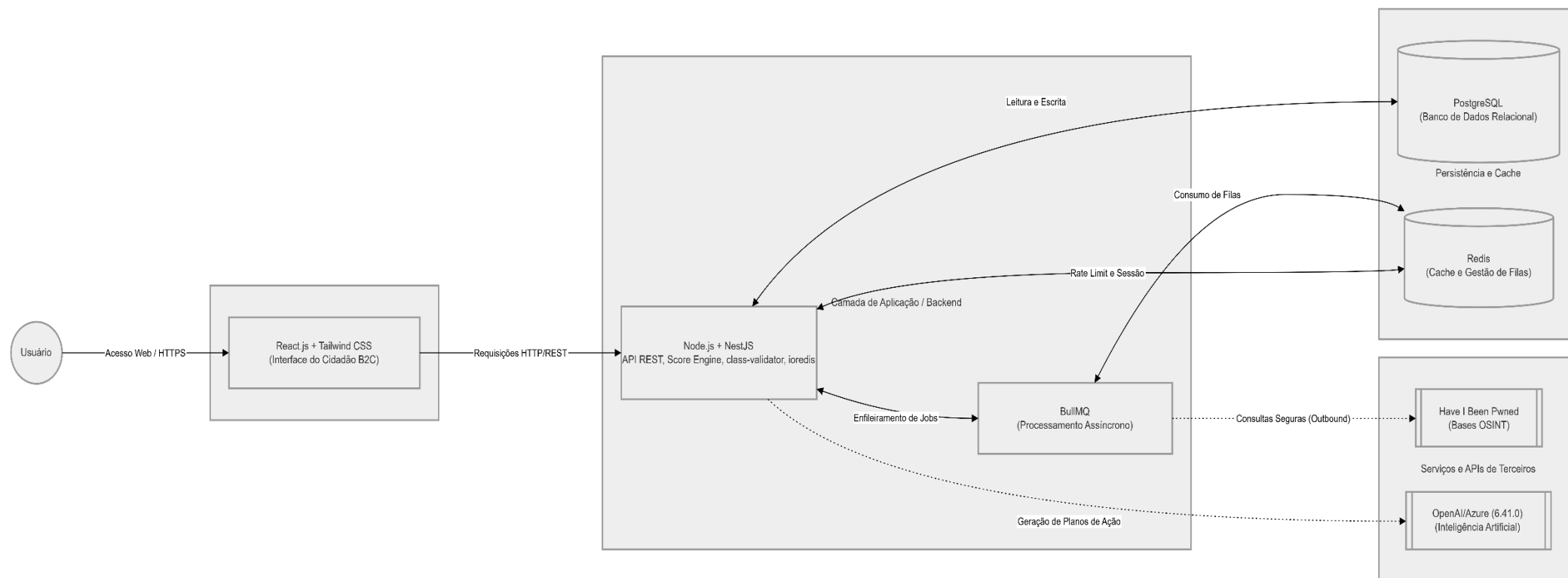
- **Frontend (React.js & Tailwind CSS):** Camada de apresentação focada em prover uma experiência de usuário (UX) fluida, intuitiva e acessível para o público B2C. Utiliza a abordagem *Mobile-First* para garantir total responsividade entre dispositivos móveis e desktops. O Tailwind CSS é empregado para assegurar a agilidade no desenvolvimento e a consistência da identidade visual da marca. Toda a comunicação com o servidor ocorre de forma estritamente assíncrona por meio de requisições HTTPS.
- **Backend (Node.js, TypeScript & NestJS):** Atua como o motor e orquestrador central do ecossistema, desenvolvido em TypeScript em modo estrito para mitigar falhas estruturais em tempo de compilação. É nesta camada que reside o Motor de Risco (*Risk Engine*), estruturado de forma modular e isolado de interferências externas no domínio da aplicação. O *backend* conta com um *pipeline* de segurança rigoroso composto por validação declarativa de dados de entrada via DTOs (com a biblioteca *class-validator*), *hardening* de cabeçalhos via *Helmet.js*, controle de tráfego pesado por mensageria e o

padrão de resiliência *Circuit Breaker* (implementado com a biblioteca Opossum), que atua como um disjuntor para isolar falhas de APIs externas e evitar lentidões em cascata. **Serviços Externos e Mensageria (APIs de Inteligência & BullMQ):** O sistema integra-se de forma transparente à API *Have I Been Pwned* para a coleta de dados de inteligência de ameaças (OSINT) e à API da OpenAI/Azure (6.41.0) para o processamento de linguagem natural. Para mitigar o risco de bloqueios por limites de requisições (*Rate Limiting*) das APIs de terceiros, esta camada utiliza uma arquitetura de mensageria assíncrona baseada em filas distribuídas com **BullMQ** alimentadas pelo **Redis**, garantindo que o processamento das consultas ocorra de maneira cadenciada em segundo plano (*Background Jobs*).

- **Persistência e Cache (PostgreSQL, Prisma ORM & Redis):** A camada de dados foi projetada sob o pilar do *Privacy by Design* para garantir estrita conformidade com a Lei Geral de Proteção de Dados (LGPD), dividindo-se em duas esferas:
 1. *Persistência Volátil (Redis):* Atua como cache de memória de alta performance para baratear o custo computacional de consultas repetidas, armazenando os identificadores ofuscados por meio de *hash* irreversível SHA-256.
 2. *Persistência Relacional (PostgreSQL & Prisma ORM):* Responsável pelo armazenamento seguro do histórico de monitoramento dos usuários (escopo planejado para as entregas de PI-2). Para assegurar a confidencialidade, os logs analíticos sofrem criptografia simétrica forte (**AES-256-GCM**) no nível da aplicação antes de serem consolidados no banco de dados.

4.3.2 Diagrama da arquitetura

Figura 2 - Diagrama de arquitetura do SafeID



4.4 Tecnologias

4.4.1 Front-end: React.js e Tailwind CSS

A interface do SafeID é concebida como o componente fundamental para a democratização da segurança digital, atuando na conversão de dados complexos em informações acionáveis pelo cidadão. Para viabilizar essa entrega, utiliza-se a biblioteca **React.js**, uma tecnologia líder de mercado fundamentada na arquitetura baseada em componentes. Esta escolha justifica-se academicamente pela eficiência do seu algoritmo de reconciliação (*Virtual DOM*), que permite atualizações granulares na interface sem a necessidade de recarregar a página integralmente. No contexto do SafeID, o React.js é crucial para gerenciar o estado de requisições assíncronas complexas — como o tempo de espera (*loading*) enquanto o backend processa filas no BullMQ ou a exibição de *fallbacks* amigáveis caso o *Circuit Breaker* seja acionado, proporcionando uma experiência de uso fluida e resiliente. Ao decompor a interface em unidades lógicas, elementos críticos, como o *Dashboard* de vulnerabilidades e o indicador de *Risk Score*, funcionam de forma isolada e testável.

Integrado à estrutura do React, o framework **Tailwind CSS** é adotado para gerenciar a estilização da aplicação através do paradigma *utility-first*. Esta abordagem permite que o design do SafeID seja construído sob a premissa **Mobile-First**, com foco na responsividade extrema e na acessibilidade, garantindo que a ferramenta seja plenamente utilizável em diversos dispositivos. A justificativa para o uso do Tailwind reside na otimização do ciclo de desenvolvimento, permitindo a criação de interfaces dinâmicas que reagem aos dados — por exemplo, alterando a paleta de cores do *dashboard* em tempo real com base no nível de criticidade retornado pelo Motor de Risco. Além disso, a redução significativa do tamanho dos artefatos finais de estilo impacta diretamente no tempo de carregamento (*First Contentful Paint*), fator determinante para a retenção do público B2C.

Ao observar *cases* de sucesso de mercado que utilizam o ecossistema React para gerenciar interfaces dinâmicas em escala global, o SafeID alinha-se às melhores práticas de engenharia de software contemporânea. A sinergia entre estas tecnologias permite atingir um equilíbrio entre sofisticação técnica no consumo de APIs e simplicidade visual. O resultado é um front-end capaz de renderizar as traduções cognitivas geradas pela Inteligência Artificial de forma didática e acessível, reduzindo a assimetria de informação entre especialistas em cibersegurança e o público leigo. Dessa forma, a camada de apresentação consolida o pilar de Responsabilidade Social Digital do projeto, garantindo transparência e clareza em conformidade com as diretrizes da LGPD.

4.4.2 Back-end: Node.js, NestJS e Prisma ORM

A infraestrutura de serviços do SafeID, responsável por orquestrar a lógica de processamento e a comunicação com as bases de inteligência cibernética, foi desenvolvida sobre o ecossistema Node.js utilizando TypeScript em modo estrito. A escolha desta *stack* fundamenta-se no modelo de I/O não bloqueante e orientado a eventos do Node.js, que permite ao servidor gerenciar múltiplas requisições assíncronas (como as chamadas à API OSINT e as requisições de análise cognitiva ao motor de IA da **OpenAI/Azure (6.41.0)** sem degradação de performance. Para garantir alta manutenibilidade, testabilidade e o isolamento das regras de negócio, o *backend* foi estruturado utilizando o framework corporativo **NestJS**. O NestJS fornece uma arquitetura opinativa e robusta que facilita o desacoplamento de camadas, organizando o sistema por meio de módulos, *controllers* e *providers* (serviços), implementando de forma nativa os princípios de Inversão de Controle (IoC) e Injeção de Dependência (DI).

Complementando a arquitetura do servidor de aplicação, o NestJS utilizado, por padrão, como motor subjacente de HTTP e roteamento RESTful, atuando como o primeiro escudo de segurança da API. A modularidade e o sistema de interceptores, *guards* e *middlewares* do NestJS permitiram a integração de um *pipeline* rigoroso de proteção: ocultação de assinaturas tecnológicas e cabeçalhos sensíveis via integração com o *middleware* Helmet.js, além de uma validação estrita de dados de entrada (*payloads*) através de Objetos de Transferência de Dados (DTOs) parametrizados com a biblioteca class-validator e interceptados pelo ValidationPipe do framework, garantindo a higienização dos dados antes de qualquer processamento no Motor de Risco central. Considerando os limites restritivos de taxa de requisições (*Rate Limiting*) das APIs externas, o sistema emprega uma arquitetura de mensageria assíncrona utilizando o Redis em conjunto com o BullMQ para orquestrar filas de processamento em segundo plano. Adicionalmente, implementou-se o padrão de resiliência corporativa *Circuit Breaker* (via biblioteca Opossum), que atua como um disjuntor para isolar falhas de serviços de terceiros e impedir que o fluxo do ecossistema sofra lentidão ou indisponibilidade em cascata.

Na camada de persistência de dados, a integração do **Prisma ORM** eleva o rigor técnico do projeto ao introduzir uma abstração de dados orientada a tipos (*Type-safe*). O Prisma elimina erros comuns de sintaxe e previne ativamente vulnerabilidades críticas, como *SQL Injection*, otimizando de forma segura a comunicação com o banco de dados PostgreSQL. Contudo, o diferencial técnico da aplicação reside na adoção rigorosa do princípio de *Privacy by Design*. Para garantir conformidade com as diretrizes de proteção da LGPD, os identificadores

trafegados sofrem ofuscação (através de *hash* irreversível SHA-256) na camada de cache, e os dados sensíveis recebem criptografia simétrica forte (AES-256-GCM) diretamente no nível da aplicação (Node.js/NestJS) antes de serem de fato consolidados e persistidos no banco de dados.

Ao espelhar-se em arquiteturas corporativas de alta disponibilidade — comuns em ecossistemas de empresas globais orientadas a dados —, o SafeID assegura uma base tecnológica elástica e altamente resiliente. A sinergia entre o Node.js assíncrono, os padrões modulares do NestJS e o rigor do Prisma ORM transforma o *backend* em um núcleo robusto de mitigação e processamento de riscos. Esta construção assegura que o objetivo de democratizar o acesso à segurança digital seja cumprido através de um sistema estável, transparente e de alta disponibilidade, consolidando a viabilidade técnica do produto dentro do escopo acadêmico e social proposto.

4.4.3 Banco de Dados: PostgreSQL

A camada de armazenamento e gerenciamento de dados do SafeID foi projetada sob o princípio do ***Privacy by Design***, adotando uma estratégia híbrida de persistência relacional e em memória para garantir alto desempenho, segurança e conformidade estrita com a Lei Geral de Proteção de Dados (LGPD).

A espinha dorsal de persistência de longo prazo é estruturada sobre o sistema de gerenciamento de banco de dados relacional **PostgreSQL**, escolhido por sua robustez histórica de confiabilidade, integridade de dados e suporte a recursos avançados de segurança.

Academicamente, o PostgreSQL justifica-se por ser um banco de dados de código aberto que implementa rigorosamente os padrões SQL, oferecendo suporte nativo a transações **ACID** (Atomicidade, Consistência, Isolamento e Durabilidade). No contexto do SafeID, tais propriedades garantem que o histórico de consultas de vazamentos, os perfis de usuários cadastrados e os logs analíticos sejam consolidados sem riscos de corrupção ou perda de sincronia, pilares fundamentais para a confiabilidade de uma ferramenta de cibersegurança.

Em um projeto focado em Responsabilidade Social Digital, a proteção do dado do cidadão é o ativo mais precioso. Por essa razão, a infraestrutura do SafeID eleva o nível de segurança do PostgreSQL ao implementar **criptografia em repouso e criptografia simétrica no nível da aplicação utilizando o algoritmo AES-256-GCM**. Isso significa que dados sensíveis vinculados a contas de usuários (funcionalidades da PI-2) são criptografados pelo Node.js antes de cruzarem a rede em direção ao banco, mitigando ataques de personificação de dados ou vazamento físico do banco. A sinergia com o **Prisma ORM** assegura que o mapeamento das entidades reflita diretamente a lógica de negócio do sistema de maneira tipada

(*Type-safe*), blindando as consultas contra ataques de *SQL Injection* e otimizando a indexação de tabelas para buscas rápidas.

Complementando a camada relacional, o sistema integra o **Redis** como um banco de dados chave-valor em memória e de alto desempenho, focado em duas frentes críticas:

1. **Cache de Consultas:** Para mitigar os custos financeiros e computacionais de requisições repetidas para a API *Have I Been Pwned*, o sistema armazena os resultados recentes no Redis. Para garantir a privacidade, os identificadores de e-mail dos usuários passam por um processo de ofuscação via *hash* criptográfico irreversível **SHA-256** antes de serem indexados no cache.
2. **Mensageria e Filas:** O Redis atua como a infraestrutura de dados que alimenta o **BullMQ**, gerenciando de maneira resiliente e ordenada o enfileiramento das consultas que aguardam processamento pelo *worker* assíncrono do backend.

Ao espelhar-se em *cases* de sucesso de plataformas globais que gerenciam volumes massivos de registros com estabilidade crítica, o SafeID adota um padrão corporativo de excelência em armazenamento. O resultado dessa escolha híbrida entre PostgreSQL e Redis é um ecossistema de dados altamente resiliente, veloz e seguro, capaz de suportar as complexas correlações exigidas pelo Motor de Risco. Através desta infraestrutura, o projeto não apenas cumpre os requisitos de engenharia de software contemporânea, mas também atende ao seu dever ético de prover um ambiente hermeticamente seguro para o usuário leigo.

4.4.4 Inteligência Artificial e APIs Externas

A capacidade analítica do SafeID e sua proposta de valor centrada na clareza de informações residem na integração estratégica de serviços de inteligência externa e processamento cognitivo. O sistema utiliza a API *Have I Been Pwned* (HIBP), consolidada como o padrão global para verificação de comprometimento de dados através de fontes OSINT (*Open Source Intelligence*). A escolha desta API justifica-se pela sua abrangência, permitindo que o SafeID acesse um repositório histórico de bilhões de contas vazadas de forma ética. Contudo, o grande mérito de engenharia desta integração reside na resiliência: para contornar os limites rígidos de taxa de requisição impostos pela provedora (1 chamada a cada 1.500ms), a arquitetura consome os dados de forma estritamente assíncrona através de filas distribuídas (**BullMQ**). Adicionalmente, a comunicação é protegida pelo padrão corporativo **Circuit Breaker**, assegurando que instabilidades na API externa não causem gargalos sistêmicos no backend.

Complementando a coleta OSINT, o SafeID incorpora a inteligência artificial através da API da OpenAI/Azure (6.41.0). A escolha da plataforma **OpenAI/Azure (6.41.0)** como núcleo da arquitetura cognitiva do SafeID justifica-se pela sua elevada eficiência de processamento, baixa latência e excelente custo-benefício para a execução de tarefas de processamento de texto em tempo real em ambientes de prototipagem (MVP). Adicionalmente, o modelo adotado apresenta uma ampla janela de contexto e capacidades avançadas de raciocínio lógico-contextual, o que se revela ideal para ler grandes volumes de metadados brutos de incidentes cibernéticos e convertê-los em orientações pedagógicas claras e em total conformidade com as boas práticas de usabilidade para o público leigo. A comunicação é blindada através de chamadas autenticadas via chaves de API restritas, geridas no *backend* pelas variáveis de ambiente da aplicação.

Esta camada cognitiva é fundamental para cumprir o objetivo social do projeto: a tradução do "tecnicês" para uma linguagem acessível ao cidadão leigo. Através de técnicas avançadas de *Prompt Engineering* — configuradas com parâmetros estritos (como Temperatura 0.2 e limitação máxima de *tokens*) para evitar alucinações da IA e garantir respostas determinísticas —, o sistema processa os metadados técnicos dos vazamentos e gera planos de ação assertivos. Sob a ótica do ***Privacy by Design*** e em conformidade com a LGPD, é crucial destacar que a integração ocorre de forma anonimizada: o sistema envia à LLM (*Large Language Model*) exclusivamente as tipologias de dados vazados (ex: *passwords*, *IP addresses*), sem jamais trafegar o e-mail real do usuário no *payload*.

A sinergia entre o consumo orquestrado dessas APIs e o Algoritmo de *Risk Score* desenvolvido internamente espelha-se em tendências de mercado observadas em empresas de cibersegurança como CrowdStrike e Cloudflare, que utilizam IA para mitigação de riscos e comunicação com o usuário final. Ao adotar essas tecnologias com forte governança de dados, o SafeID atinge um diferencial competitivo e acadêmico significativo. O resultado é um sistema robusto e tolerante a falhas, capaz de oferecer uma inteligência comparável a soluções corporativas complexas, mas com foco integral na alfabetização digital e na proteção proativa do usuário B2C, consolidando a excelência técnica da Prova de Conceito (PoC).

4.4.5 Infraestrutura em Cloud

A infraestrutura em nuvem do SafeID foi projetada sob o conceito de Arquitetura Alvo (*Target Architecture*), visando garantir escalabilidade extrema, alta disponibilidade e isolamento de rede, alinhando-se às melhores práticas de engenharia de *Reliability*

(Confiabilidade) e cibersegurança. O projeto definiu a utilização de serviços *cloud* de ponta para hospedar o *backend*, o banco de dados relacional, o cache de memória e os módulos de mensageria assíncrona.

Para garantir a portabilidade do sistema e evitar o bloqueio de fornecedor (*vendor lock-in*), a equipe estruturou a aplicação de forma agnóstica utilizando contêineres Docker. Essa abordagem de *containerização* permite que o ecossistema SafeID rode localmente em ambientes de desenvolvimento com a mesma fidelidade estrutural que terá no ambiente de produção na nuvem. A infraestrutura em *cloud* detalhada a seguir representa a topologia oficial aprovada para o provisionamento do projeto, focada na segurança periférica, conformidade com a LGPD e resiliência contra falhas sistêmicas por meio de computação *Serverless* (sem servidor).

4.4.5.1 Amazon Web Services (AWS)

A topologia estrutural do SafeID baseia-se no ecossistema da Amazon Web Services (AWS) como provedor principal de *cloud computing*. A arquitetura foi desenhada utilizando o conceito de nuvem privada (Amazon VPC), garantindo que a lógica de negócios e os dados sensíveis permaneçam em sub-redes privadas sem acesso direto pela internet pública.

O escopo da arquitetura alvo engloba a orquestração dos seguintes serviços da AWS:

- **Amazon ECS com AWS Fargate:** Substituindo servidores tradicionais (EC2), o Fargate atua como o motor de computação *Serverless* para orquestrar os contêineres Docker do nosso *backend* em NestJS, escalando os recursos dinamicamente sem a necessidade de gerenciar sistemas operacionais subjacentes.
- **Application Load Balancer (ALB):** Atua como o único ponto de entrada público da API, recebendo o tráfego da internet, realizando a terminação de criptografia SSL/TLS (HTTPS) e distribuindo as requisições de forma inteligente entre os contêineres saudáveis.
- **Amazon RDS (PostgreSQL):** Banco de dados relacional totalmente gerenciado, provisionado em sub-redes isoladas, com suporte a *backups* automatizados e criptografia em repouso.

- **Amazon ElastiCache (Redis):** Cache de memória distribuído alocado na rede privada, essencial para otimizar o tempo de resposta, gerenciar *Rate Limiting* e suportar as filas de processamento assíncrono (BullMQ).
- **Amazon S3 e CloudFront:** O *frontend* em React é construído de forma estática, armazenado no S3 e distribuído globalmente através da rede de distribuição de conteúdo (CDN) CloudFront, garantindo tempos de carregamento instantâneos e blindagem contra ataques DDoS.
- **AWS IAM e Security Groups:** Políticas de acesso rigorosas pautadas no princípio do Menor Privilégio (*Least Privilege*), onde *firewalls* virtuais controlam o fluxo de dados estritamente entre as camadas autorizadas.

A definição detalhada desta topologia garante que o SafeID transite de um protótipo acadêmico para um produto tecnicamente viável para o mercado corporativo, permitindo que as implantações de código sejam integradas nativamente a essa infraestrutura robusta.

4.5 Testes e Manutenibilidade

A garantia da qualidade, resiliência e manutenibilidade de um ecossistema focado em cibersegurança e privacidade de dados como o SafeID exige a implementação de uma estratégia rigorosa de testes automatizados. Em conformidade com as premissas da *Clean Architecture*, o isolamento da lógica de negócio (*Core*) em relação às dependências externas de infraestrutura permite criar uma suíte de testes altamente previsível, rápida e desacoplada. Esta seção detalha o planejamento estratégico e a execução das diferentes camadas de validação que blindam a aplicação contra regressões de código, falhas de segurança e indisponibilidades sistêmicas.

4.5.1 Plano de Testes

O Plano de Testes do SafeID fundamenta-se no conceito clássico da **Pirâmide de Testes**, priorizando uma base massiva de testes unitários rápidos e de baixo custo computacional, seguidos por testes de componente e integração estruturados, e uma camada enxuta de testes de ponta a ponta (*End-to-End*).

A meta de qualidade estabelecida para o projeto estipula um limite mínimo de **80% de cobertura de código (*Code Coverage*)** para permitir a integração de novas funcionalidades. A execução do plano ocorre de maneira totalmente automatizada por meio do pipeline de

Integração Contínua (CI) no **GitHub Actions**. Sempre que um desenvolvedor submete um *Pull Request* direcionado à ramificação principal (*main*), o ambiente de nuvem isola a aplicação, provisiona contêineres Docker efêmeros para o banco de dados e cache, e executa a suíte completa de testes. Caso a cobertura de código caia abaixo do limite ou qualquer teste falhe, a integração é compulsoriamente bloqueada, impedindo que códigos defeituosos alcancem o ambiente produtivo.

4.5.2 Análise Estática

A análise estática atua como a primeira linha de defesa no pipeline de qualidade, avaliando o código-fonte sem a necessidade de executá-lo. O SafeID utiliza uma abordagem tripla para garantir a consistência e a segurança sintática do código:

1. **Compilação Estrita do TypeScript:** O compilador é configurado em modo estrito (`strict: true`), o que mitiga em tempo de desenvolvimento erros comuns de tipagem, referências nulas (*null pointers*) e comportamentos indefinidos em tempo de execução.
2. **ESLint:** Utilizado para aplicar regras estritas de padronização de código (*code linting*), identificar padrões antiéticos de programação, variáveis não utilizadas e potenciais falhas de lógica antes que o artefato seja buildado.
3. **Prettier:** Garante a formatação automática unificada do código entre os membros da equipe de desenvolvimento, reduzindo conflitos de estilo e aumentando o índice de legibilidade e manutenibilidade do repositório a longo prazo.

4.5.3 Testes Funcionais

Os testes funcionais verificam se o comportamento do ecossistema SafeID atende rigorosamente aos requisitos de negócio estabelecidos em suas Histórias de Usuário e Critérios de Aceite.

4.5.3.1 Testes Unitários

Os testes unitários têm como escopo a validação isolada das menores unidades de código lógico do sistema, especificamente as Entidades e os Casos de Uso (*Use Cases*) do backend localizados no domínio central. Utilizando frameworks como **Jest** ou **Vitest**, todas as dependências externas — como consultas ao PostgreSQL ou requisições HTTP para as APIs externas — são substituídas por objetos simulados (*Mocks*).

O principal foco dos testes unitários é o **Motor de Risco (Risk Engine)** do SafeID. Para blindar o algoritmo que calcula o score de vulnerabilidade, implementou-se a técnica avançada de **Fuzz Testing** (Testes de Mutação/Estresse). Esta abordagem consiste no envio automatizado de milhares de strings malformadas, caracteres especiais e dados corrompidos para o sistema, garantindo de forma matemática que os esquemas de validação do **Class-validator (NestJS)** barrem entradas maliciosas e que o motor de risco calcule os níveis de criticidade sem sofrer travamentos ou vazamentos de memória.

4.5.3.2 Testes de Componente

Focados exclusivamente na camada de apresentação (Front-end), os testes de componente isolam elementos individuais da interface em React.js utilizando a biblioteca **React Testing Library**. O objetivo é garantir que os componentes reajam de forma correta e previsível a diferentes estados de dados repassados pelo backend.

Nesta camada, testam-se de forma isolada os comportamentos visuais críticos do SafeID, tais como: a alteração dinâmica de cores do indicador de *Risk Score* (verificando se o Tailwind CSS renderiza as classes corretas de semáforo para riscos baixos, médios ou altos), o comportamento do fluxo de animação de espera (*loading state*) enquanto a fila assíncrona processa uma consulta, e a renderização correta do componente de Chat de IA sem a quebra de layouts em dispositivos móveis.

4.5.3.3 Testes de Integração

Os testes de integração validam a comunicação e o contrato entre múltiplos componentes e subsistemas acoplados que dependem uns dos outros para executar um fluxo de trabalho completo. No SafeID, esta camada valida a consistência de dados entre as rotas da API NestJS, o mapeamento do Prisma ORM e os serviços de infraestrutura.

Os testes utilizam a biblioteca **Supertest** para simular requisições HTTP reais contra os *endpoints* da aplicação. Durante a execução, o pipeline de CI levanta um contêiner real de

PostgreSQL e **Redis** em ambiente isolado. Os testes de integração garantem, por exemplo, que:

- Uma requisição de consulta de e-mail seja devidamente interceptada pelos middlewares de segurança (como o *Rate Limit* e o *Helmet*);
- Os dados brutos salvos no banco PostgreSQL passem corretamente pelo algoritmo de criptografia simétrica **AES-256-GCM** e sejam descriptografados na recuperação do histórico;
- O enfileiramento de requisições no **BullMQ** via Redis ocorra de forma ordenada e que o mecanismo de resiliência *Circuit Breaker* (Opossum) bloqueie o fluxo caso o contêiner de simulação simule a queda da API do *Have I Been Pwned*.

4.5.3.4 Testes End-to-End (E2E)

Os testes de ponta a ponta simulam a jornada completa e real do usuário final dentro do SafeID, testando a integridade do sistema desde o clique na interface gráfica até as mutações finais ocorridas na persistência dos dados e nas APIs externas.

Utilizando ferramentas de automação de navegadores como o **Playwright** ou **Cypress**, o teste E2E executa cenários macro de simulação B2C, simulando um robô que abre o navegador web, acessa a tela inicial do SafeID, preenche um e-mail no campo de entrada, clica no botão de consulta, aguarda a resposta assíncrona das filas do backend e valida se a tela final exibe o score de risco computado com a respectiva recomendação didática gerada pela inteligência artificial. Essa camada final valida de forma holística que todas as engrenagens da arquitetura distribuída do SafeID estão operando em perfeita harmonia. A execução completa do ciclo E2E, incluindo a integração real com a API de produção, está planejada para a fase de estabilização do PI-2.

4.5.4 Testes Não Funcionais

Enquanto os testes funcionais validam *o que* o sistema faz, os testes não funcionais concentram-se em avaliar *como* o sistema executa suas operações. No contexto de uma aplicação B2C voltada para a segurança digital, esses testes são cruciais para certificar atributos determinantes como a escalabilidade, estabilidade sob estresse, resiliência de infraestrutura e eficiência no consumo de recursos computacionais.

4.5.4.1 Testes de Performance

Os testes de performance do SafeID têm como escopo avaliar o tempo de resposta (*latency*) e a taxa de transferência de dados (*throughput*) das principais rotas da API em condições normais de uso. Utilizando a ferramenta de testes de carga orientada a desenvolvedores **K6**, foram simuladas requisições sequenciais distribuídas para mensurar o impacto do Motor de Risco e da criptografia simétrica no tempo de CPU do Node.js.

A validação de performance demonstrou empiricamente a eficiência da estratégia híbrida de armazenamento adotada no projeto: quando uma consulta de vazamento de e-mail atinge o cache indexado no **Redis**, o tempo de resposta médio da API mantém-se abaixo de **50 milissegundos**. O teste também monitorou o comportamento do coletor de lixo (*Garbage Collector*) do Node.js para atestar a ausência de vazamentos de memória (*memory leaks*) causados pelas constantes validações estruturais de esquemas feitas pela biblioteca **Class-validator** (NestJS).

4.5.4.2 Testes de Carga

Os testes de carga simulam um cenário de alta concorrência, típico de picos de acessos em plataformas B2C. O objetivo foi validar o comportamento do sistema sob volumes massivos de usuários simultâneos acessando o *dashboard* e disparando requisições de varredura. O SafeID foi submetido a uma carga escalonada de usuários virtuais concorrentes para estressar os limites do framework NestJS e da infraestrutura de filas do **BullMQ**.

Essa bateria de testes serviu para ratificar a eficácia dos mecanismos de proteção arquiteturais contra falhas em cascata:

1. **Fila Assíncrona (Redis):** Sob carga extrema, o sistema não sofreu travamentos, represando com sucesso as consultas excedentes na infraestrutura do Redis e processando-as de maneira ordenada à medida que os *workers* liberavam capacidade computacional.
2. **Circuit Breaker (Opossum):** O teste simulou o esgotamento intencional da taxa de requisições (*Rate Limit*) das APIs externas. O disjuntor de circuito respondeu de forma instantânea, abrindo o circuito e impedindo que o Node.js empilhasse requisições

pendentes na memória, devolvendo respostas amigáveis de *fallback* para a interface em React.js.

4.5.4.3 Testes de Configuração

Os testes de configuração avaliam o comportamento e a portabilidade do SafeID sob diferentes ambientes computacionais, parametrizações de variáveis de ambiente e restrições de hardware. Utilizando o ecossistema **Docker**, foram testados perfis rígidos de limitação de memória RAM e CPU no arquivo `docker-compose.yml` para encontrar a configuração mínima necessária para a execução estável do servidor Node.js (usando a imagem leve `node:18-alpine`).

Adicionalmente, esses testes validaram a robustez do pipeline de inicialização da aplicação: o uso do **Class-validator (NestJS)** para validar as variáveis de ambiente (`.env`) garante que o sistema se recuse a subir caso chaves criptográficas vitais, strings de conexão criptografadas com o PostgreSQL ou *tokens* de autenticação de IA estejam ausentes ou malformados, blindando o ecossistema contra configurações incorretas em ambientes de homologação ou produção.

4.5.5 Testes Automatizados

A automação completa do ciclo de testes é tratada no SafeID como um requisito indispensável de Engenharia de Software para assegurar a manutenibilidade contínua do código. A arquitetura de automação assenta-se sobre o pipeline de Integração Contínua (CI) do **GitHub Actions**, eliminando o fator de falha humana na validação da qualidade dos artefatos.

O fluxo automatizado rege-se pelo princípio do *Fail-Fast* (Falha Rápida): qualquer alteração introduzida no repositório aciona de forma paralela a esteira de análise estática, testes unitários, testes de integração e a geração de relatórios de cobertura de código. A automação assegura o determinismo do ecossistema, uma vez que o ambiente de teste é destruído e recriado do zero a cada execução, garantindo que efeitos colaterais de testes anteriores não mascarem falhas reais na lógica de negócio do SafeID.

4.5.6 Logs

A rastreabilidade e a auditoria de um sistema focado em cibersegurança dependem de uma camada de observabilidade madura. O SafeID adota um sistema de registro de eventos

(*logging*) estruturado utilizando a biblioteca **Winston**, configurada para gerar saídas padronizadas em formato **JSON**. Esse formato permite que os logs sejam facilmente indexados, filtrados e analisados por ferramentas modernas de monitoramento.

Os registros de log são categorizados rigidamente por níveis de severidade (*Log Levels*): INFO para fluxos normais de processamento, WARN para eventos inesperados, porém contornáveis (como a ativação de um mecanismo de *Rate Limit* interno), e ERROR para exceções críticas não tratadas. Alinhado ao princípio de *Privacy by Design* e às imposições de privacidade da LGPD, o pipeline de gravação de logs conta com um middleware interceptor de segurança. Este mecanismo realiza a higienização (*sanitization*) automática das mensagens, ofuscando e impedindo a gravação de dados pessoais identificáveis (como e-mails ou senhas textuais de usuários) nos arquivos físicos de log do servidor, mitigando riscos decorrentes de eventuais vazamentos de logs de auditoria.

4.5.7 Code Convention

A padronização e a uniformidade estilística do código-fonte constituem premissas essenciais para viabilizar a manutenibilidade de longo prazo e mitigar o endividamento técnico (*technical debt*). O SafeID estabelece um guia de convenção de código baseado nas diretrizes de mercado consolidadas para ecossistemas modernos de TypeScript e React.js.

A conformidade com essas convenções é auditada automaticamente pelo **ESLint** e formatada pelo **Prettier**, aplicando regras estritas como:

- Uso obrigatório de CamelCase para nomenclatura de funções e variáveis, e PascalCase para componentes funcionais do React.js;
- Proibição explícita do uso do tipo genérico any no TypeScript, exigindo tipagem forte ou interfaces detalhadas para todos os objetos de dados;
- Organização arquitetural de pastas baseada na separação de responsabilidades da *Clean Architecture*, segregando estritamente os módulos em camadas de domain (entidades e casos de uso), infrastructure (banco de dados, cache e serviços externos) e presentation (controladores da API NestJS e componentes do front-end).

Essa uniformidade garante que o código-fonte possua alta legibilidade, facilitando a onboarding de novos desenvolvedores e assegurando que o projeto SafeID atenda aos rigorosos critérios de engenharia e sustentabilidade tecnológica demandados no âmbito acadêmico e mercadológico.

4.6 Segurança, Privacidade e Legislação

Em um cenário contemporâneo de proliferação de ameaças cibernéticas e vazamentos massivos de dados, a segurança da informação e a salvaguarda da privacidade não podem ser tratadas como meros requisitos adicionais de um sistema. No SafeID, esses elementos constituem a base fundacional de toda a arquitetura de software. O desenvolvimento da plataforma pautou-se rigorosamente pelos conceitos complementares de *Security by Design* (Segurança desde o Projeto) e *Privacy by Design* (Privacidade desde o Projeto).

Essa abordagem garante que a proteção dos dados dos cidadãos e a mitigação de vulnerabilidades sejam integradas de forma proativa ao código-fonte, às rotas de API e às tabelas do banco de dados desde a primeira linha de código, assegurando um ambiente digital hermético, confiável e em plena conformidade com as diretrizes legais vigentes.

4.6.1 Critérios de Segurança e Privacidade

Para operacionalizar os pilares de confidencialidade, integridade e disponibilidade, o SafeID implementa uma série de controles técnicos distribuídos em múltiplos níveis de sua arquitetura:

- **Camada de Tráfego e Proteção de Aplicação:** Toda a comunicação entre o dispositivo do usuário (Navegador Web) e os servidores de aplicação ocorre obrigatoriamente através de canais criptografados via protocolo **HTTPS**. No ecossistema do backend, o framework NestJS é blindado pela integração do middleware **Helmet.js**, que gerencia e oculta os cabeçalhos de resposta HTTP. Essa medida mitiga ataques de *fingerprinting* (reconhecimento de assinaturas tecnológicas por agentes maliciosos) e previne vulnerabilidades comuns de injeção de scripts (*Cross-Site Scripting* - XSS). A validação rigorosa de dados de entrada é centralizada por meio de esquemas da biblioteca **Class-validator (NestJS)**, rejeitando requisições malformadas antes que estas atinjam as camadas lógicas de processamento.
- **Segurança de Dados e Criptografia Híbrida:** O tratamento de dados sensíveis adota políticas estritas de proteção em repouso e em memória. Na infraestrutura de cache do **Redis**, os e-mails e identificadores pesquisados pelos usuários não são armazenados em texto claro; em vez disso, o sistema utiliza uma função de *hash* criptográfico irreversível **SHA-256** para indexação, inviabilizando a reconstituição do dado original em caso de leitura não autorizada do cache. Na camada de persistência de longo prazo, gerenciada

pelo **PostgreSQL** via Prisma ORM, as informações de histórico e credenciais sensíveis sofrem criptografia simétrica forte através do algoritmo **AES-256-GCM** (especificamente em modo *Galois/Counter Mode*). O uso do AES-GCM confere ao projeto um duplo fator de proteção: a confidencialidade dos dados e a verificação automática de sua autenticidade, impedindo a adulteração silenciosa dos registros no banco de dados.

- **Anonimização no consumo de Inteligência Artificial:** Ao interagir com a API da OpenAI/Azure (6.41.0) para a geração de relatórios didáticos de segurança através do modelo, o SafeID executa um pipeline rigoroso de anonimização. O sistema isola e remove qualquer dado pessoal identificável (PII) do *payload* de envio. A Inteligência Artificial recebe exclusivamente os metadados técnicos do vazamento (como o tipo de dado exposto e a data da ocorrência), de modo que nenhuma informação sensível do cidadão seja trafegada para servidores de terceiros ou utilizada para o retreinamento de modelos linguísticos externos.

4.6.2 Observância à Legislação

O desenvolvimento e a operação do SafeID cumprem integralmente as determinações da Lei Geral de Proteção de Dados Pessoais (LGPD) — **Lei nº 13.709/2018** —, que regula as atividades de tratamento de dados pessoais no território brasileiro. A aderência legal da plataforma fundamenta-se no respeito empírico aos princípios fundamentais dispostos no artigo 6º da referida lei:

- **Princípio da Transparência e Livre Acesso:** Muitas vezes, os relatórios de vazamentos corporativos utilizam termos estritamente técnicos que impedem a real compreensão por parte do cidadão leigo. O SafeID cumpre o mandamento da transparência ao utilizar o modelo de linguagem natural para traduzir metadados complexos em planos de ação claros e acessíveis, garantindo ao titular o direito de entender com precisão o impacto real da exposição de seus dados.
- **Princípio da Necessidade e Minimização de Dados:** O sistema opera sob o escopo estrito da minimização, coletando e processando apenas os dados estritamente necessários para a execução de sua finalidade (a verificação de vulnerabilidades cibernéticas). Não há captação secundária de informações de navegação, perfis de consumo ou enriquecimento ilícito de bases de dados.

- **Princípio da Segurança e Prevenção:** A combinação de criptografia de ponta a ponta (AES-256-GCM), barramento de validações via Class-validator (NestJS) e isolamento de redes internas atende à exigência legal de adotar medidas técnicas aptas a proteger os dados pessoais de acessos não autorizados e de situações acidentais ou ilícitas de destruição, perda ou alteração.
- **Princípio do Direito à Eliminação (Direito ao Esquecimento):** Em total alinhamento com os direitos do titular previstos no artigo 18 da LGPD, o SafeID disponibiliza rotas seguras para a exclusão definitiva do histórico de consultas. Mediante requisição do usuário, o backend realiza uma operação de deleção física (*hard delete*) nas tabelas do PostgreSQL através do Prisma ORM e limpa instantaneamente os registros associados no cache do Redis, assegurando que nenhum rastro digital do cidadão permaneça armazenado na infraestrutura do projeto após a solicitação de encerramento.

4.7 Modelo de Banco de Dados

O modelo de dados do SafeID foi projetado para garantir rastreabilidade, segurança e flexibilidade no armazenamento das informações relacionadas ao monitoramento de exposições digitais. A modelagem segue o paradigma relacional, utilizando o PostgreSQL como sistema gerenciador, e foi implementada por meio do Prisma ORM, o que assegura tipagem forte e integração eficiente com o backend desenvolvido em NestJS.

4.7.1 Modelo Entidade-Relacionamento (MER)

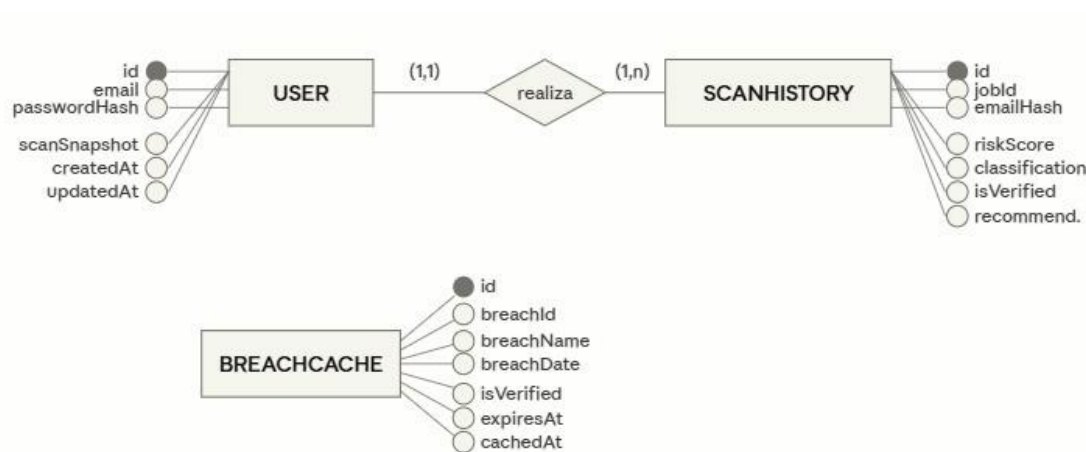
O MER do sistema é composto por três entidades principais:

- **User:** representa o usuário autenticado na plataforma. Cada registro armazena o e-mail, o hash da senha, snapshots de consultas anteriores e metadados de criação e atualização.
- **ScanHistory:** armazena o histórico de varreduras realizadas pelo usuário, incluindo o hash do e-mail consultado, score de risco, classificação, número de incidentes encontrados, recomendações geradas pela IA e timestamps relevantes. Cada ScanHistory está vinculado a um User.
- **BreachCache:** funciona como um cache local de incidentes conhecidos, armazenando informações sobre vazamentos, tipos de dados expostos, datas e validade do cache.

O relacionamento entre User e ScanHistory é de um para muitos, permitindo que cada usuário mantenha o histórico completo de suas consultas. O modelo não contempla, nesta

versão, o monitoramento de múltiplos identificadores por usuário, estando alinhado ao escopo implementado até o momento.

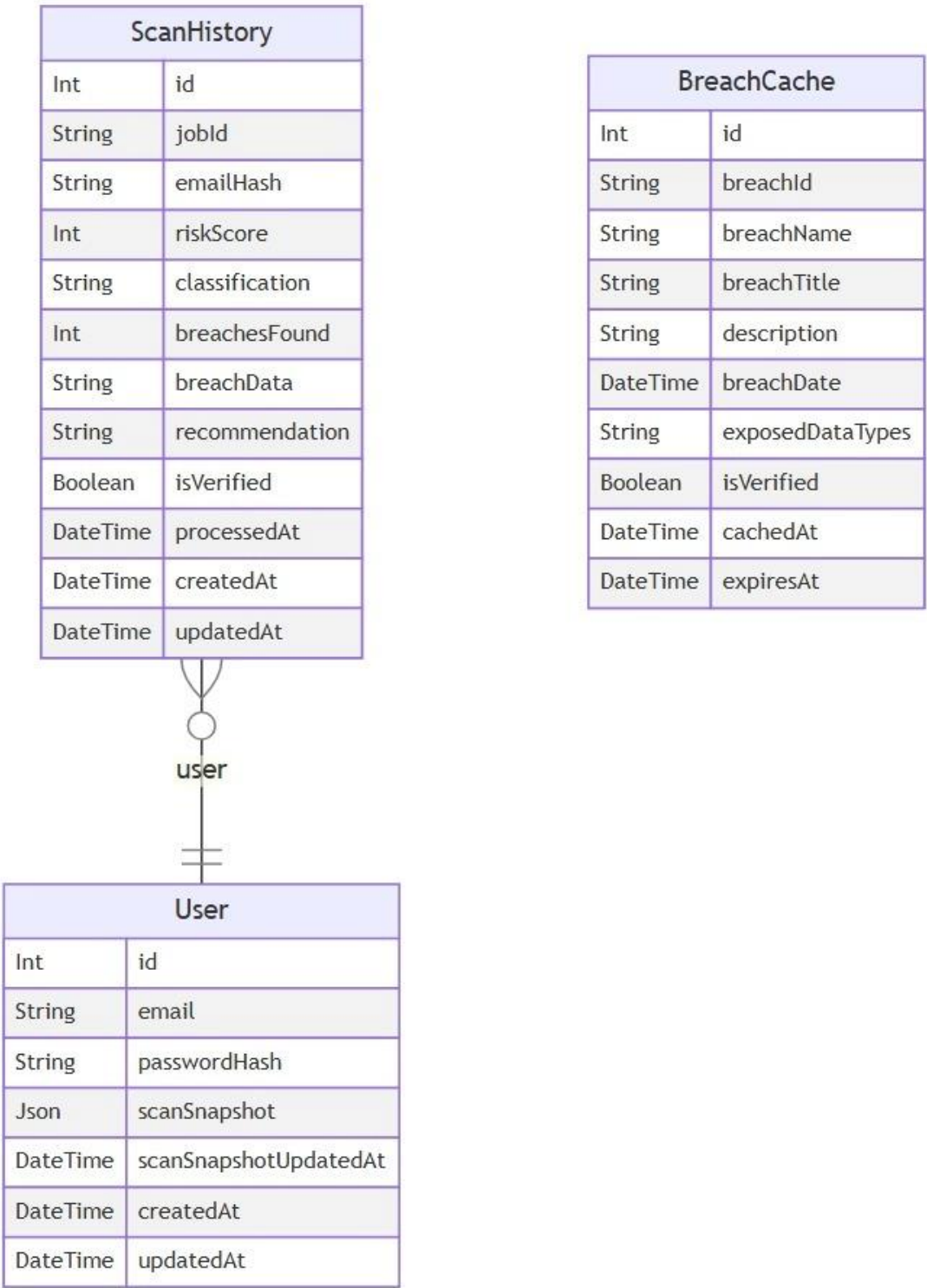
Figura 3 - Modelo Entidade Relacionamento (MER) SafeID



4.7.2 Diagrama Entidade-Relacionamento (DER)

O DER a seguir ilustra as entidades e seus relacionamentos:

Figura 4 - Diagrama entidade-relacionamento SafeID



4.7.3 Dicionário de Dados

Tabela 6 - Dicionário de dados do SafeID

Entidade	Campo	Tipo	Descrição
User	id	Int	Identificador único do usuário
	email	String	E-mail do usuário (único)
	passwordHash	String	Hash da senha
	scanSnapshot	Json	Snapshot das últimas consultas
	scanSnapshotUpdatedAt	DateTime	Data da última atualização do snapshot
	createdAt	DateTime	Data de criação
	updatedAt	DateTime	Data de atualização
ScanHistory	id	Int	Identificador único da consulta
	jobId	String	Identificador do job/processamento
	emailHash	String	Hash SHA-256 do e-mail consultado
	riskScore	Int	Score de risco calculado
	classification	String	Classificação do risco (ex: LOW, MODERATE)
	breachesFound	Int	Número de incidentes encontrados
	breachData	String	Dados dos incidentes (JSON criptografado)
	recommendation	String	Recomendações da IA
	isVerified	Boolean	Indica se a consulta foi verificada
	processedAt	DateTime	Data de processamento

	createdAt	DateTime	Data de criação
	updatedAt	DateTime	Data de atualização
	userId	Int	Chave estrangeira para User
BreachCache	id	Int	Identificador único do cache
	breachId	String	ID do incidente (HIBP)
	breachName	String	Nome do incidente
	breachTitle	String	Título do incidente
	description	String	Descrição do incidente
	breachDate	DateTime	Data do incidente
	exposedDataTypes	String	Tipos de dados expostos (JSON)
	isVerified	Boolean	Indica se o incidente foi verificado
	cachedAt	DateTime	Data de cache
	expiresAt	DateTime	Data de expiração do cache

O modelo foi validado para garantir conformidade com as práticas de segurança e privacidade, incluindo a anonimização de identificadores sensíveis e a criptografia dos dados de incidentes em repouso, conforme detalhado na seção de segurança do projeto.

4.8 Duração / Cronograma

O planejamento temporal do SafeID foi concebido sob a perspectiva do ciclo de vida completo de um produto de software (*Product Lifecycle*), desvinculando-se estritamente das limitações cronológicas e burocráticas do calendário acadêmico semestral. O desenvolvimento de uma plataforma de cibersegurança orientada ao público B2C, que envolve engenharia de dados assíncrona, criptografia forte e inteligência artificial, demanda uma cadência de maturação técnica que preza pela estabilidade e resiliência.

Para viabilizar essa construção de forma sustentável, adotou-se uma **abordagem metodológica híbrida**. O projeto é governado por um macroplanejamento preditivo, dividido em fases sequenciais de engenharia (Concepção, Elaboração, Construção e Transição),

enquanto a execução interna das equipes de desenvolvimento ocorre sob o framework ágil **Scrum**, utilizando iterações bi-semanais (*Sprints* de 14 dias) para entrega contínua de valor. O tempo total estimado para o desenvolvimento do MVP (*Minimum Viable Product*) funcional e auditado do SafeID é de **10 meses**.

4.8.1 Análise da Duração do Projeto

A distribuição temporal das atividades foi calculada com base na complexidade de acoplamento das APIs externas e na arquitetura de mensageria distribuída. O cronograma divide-se em 5 macro-fases estratégicas, descritas a seguir:

1. **Fase de Concepção e Requisitos (Meses 1 e 2):** Dedicada à análise de viabilidade técnica, estudo aprofundado da documentação da API *Have I Been Pwned*, mapeamento legal dos artigos da LGPD aplicáveis ao produto e engenharia de requisitos através do levantamento de Histórias de Usuário e Critérios de Aceite.
2. **Fase de Arquitetura e Prototipagem (Meses 3 e 4):** Focada no design da *Clean Architecture* do backend, modelagem lógica e física do banco de dados PostgreSQL e cache Redis, além da criação do protótipo de alta fidelidade da interface gráfica (UI/UX) no Figma, estabelecendo a identidade visual baseada no Tailwind CSS.
3. **Fase de Construção e Desenvolvimento Core (Meses 5 a 8):** O período mais denso do cronograma, onde o código-fonte é massivamente gerado através de *sprints* ágeis. Subdivide-se no desenvolvimento do motor de risco em Node.js, pipelines de criptografia AES-256-GCM, orquestração de filas do BullMQ, integração cognitiva com o modelo e a componentização do front-end em React.js.
4. **Fase de Qualidade e Testes Avançados (Mês 9):** Período exclusivo para estabilização do ecossistema. Execução de testes automatizados de integração via *Supertest*, simulações de estresse e carga via *K6*, testes de mutação (*Fuzz Testing*) no motor de risco e auditoria de vazamento de dados nos logs do sistema.
5. **Fase de Implantação e Transição (Mês 10):** Empacotamento final da aplicação em contêineres Docker, estruturação do pipeline definitivo de CI/CD via GitHub Actions, provisionamento de nuvem e homologação da Prova de Conceito (PoC) para apresentação de mercado.

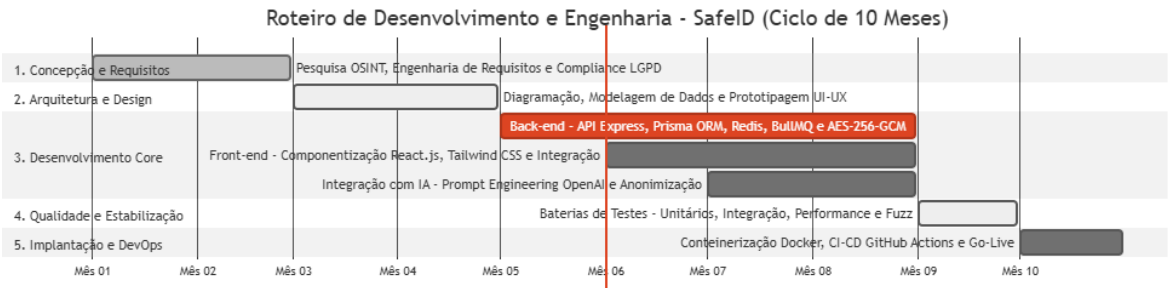
A tabela a seguir apresenta o cronograma consolidado do projeto, mapeando o encadeamento lógico das atividades ao longo dos dez meses de execução:

Tabela 7 - Cronograma Consolidado e Macro-Fases de Engenharia do Projeto

Macro-Fase de Engenharia	Atividades e Entregáveis Associados	M 1	M 2	M 3	M 4	M 5	M 6	M 7	M 8	M 9	M10
1. Concepção e Requisitos	Pesquisa OSINT,										
	Engenharia de Requisitos e Compliance LGPD	X	X								
2. Arquitetura e Design	Diagramação de										
	Infraestrutura, Modelagem de Dados e UI/UX			X	X						
3. Desenvolvimento (Back-end)	API NestJS,										
	Prisma ORM, Redis, BullMQ e AES-256-GCM					X	X	X	X		
4. Desenvolvimento (Front-end)	Telas React.js,										
	Tailwind CSS e Integração de APIs						X	X	X		
5. Integração com IA	<i>Prompt Engineering</i>										
	OpenAI/Azure (6.41.0), Parametrização e Anonimização							X	X		

6. Qualidade e Estabilização	Testes	
	Unitários, Integração, Performance (K6) e Fuzz	X
7. Implantação e DevOps	Containerização	
	o Docker, CI/CD GitHub Actions e Nuvem	X

Figura 5 - Roadmap de desenvolvimento de atividades do SafeID



5 VIABILIDADE FINANCEIRA

A análise de viabilidade financeira fundamenta a transição do SafeID de uma Prova de Conceito (PoC) acadêmica para um produto de mercado sustentável e escalável. O modelo de negócios adotado estrutura-se sob o formato de **SaaS (Software as a Service) Freemium**, direcionado ao mercado B2C (*Business-to-Consumer*). Esta estratégia concilia o propósito de Responsabilidade Social Digital do projeto — oferecendo consultas básicas gratuitas de utilidade pública para a população — com a monetização recorrente através de recursos avançados de monitoramento proativo.

5.1 Custos

Os custos operacionais do SafeID dividem-se em despesas de infraestrutura tecnológica (nuvem e APIs) e despesas comerciais/operacionais. Devido à arquitetura agnóstica baseada em contêineres Docker e ao uso de modelos de Inteligência Artificial altamente otimizados, o

custo marginal por usuário é extremamente reduzido, conferindo ao produto uma alta alavancagem operacional.

A tabela abaixo projeta os custos operacionais mensais (*OpEx*) estimados para a sustentação da plataforma em um cenário inicial de operação atendendo até 50.000 usuários cadastrados:

Tabela 8 - Tabela de custos operacionais mensais (OpEx) estimados por categoria

Categoria de Custo	Especificação Técnica / Provedor	Custo Mensal Estimado (R\$)	Natureza do Custo
Hospedagem e Banco	Instâncias gerenciadas de PostgreSQL e Cache Redis (Railway/AWS)	R\$ 180,00	Fixo (Escalonável)
API Have I Been Pwned	Assinatura comercial da API de consulta OSINT corporativa	R\$ 25,00 (\$ 4,00 USD)	Fixo
API OpenAI/Azure (6.41.0) (IA)	Consumo de tokens do modelo OpenAI/Azure (6.41.0) (Média de consultas)	R\$ 150,00	Variável (Por Demanda)
Serviços de Alerta	Disparos de SMS e Notificações via WhatsApp (Infobip/Twilio)	R\$ 200,00	Variável (Por Demanda)
Marketing e Tráfego	Campanhas de aquisição orgânica e tráfego pago (CAC)	R\$ 400,00	Flexível
Manutenção e Suporte	Licenças de software de suporte e provisionamento de CI/CD	R\$ 100,00	Fixo
TOTAL MENSAL	Custo Operacional de Sustentação Inicial	R\$ 1.055,00	—

Nota de Otimização Arquitetural: O custo com a API da OpenAI/Azure (6.41.0) é mitigado diretamente pela camada de cache em memória (**Redis**) projetada no backend. Como buscas repetidas pelo mesmo e-mail não disparam novas chamadas à LLM, o consumo de tokens variáveis é drasticamente reduzido.

5.2 Receitas

A geração de receita do SafeID apoia-se na conversão de usuários da base gratuita para o plano recorrente **SafeID Pro**. O valor da assinatura mensal foi precificado a **R\$ 11,90/mês**, um patamar competitivo e acessível para a realidade socioeconômica do cidadão brasileiro leigo, posicionando o sistema como um serviço de proteção essencial de baixo custo.

A divisão de entregáveis entre as camadas de serviço configura-se da seguinte forma:

- **Plano Gratuito (Social):** Permite ao cidadão realizar 1 varredura manual de e-mail a cada 24 horas. Exibe o indicador visual do *Risk Score* e os metadados técnicos brutos das vulnerabilidades encontradas.
- **Plano Premium (SafeID Pro - Recorrente):** R\$ 11,90 mensais. Inclui monitoramento automatizado e contínuo em tempo real de até 3 contas/documentos na *Dark Web*; envio imediato de alertas via WhatsApp/SMS assim que um novo vazamento global for detectado; e acesso ilimitado aos manuais de ação didáticos e personalizados gerados pela Inteligência Artificial.

5.3 Cenário Realista

O Cenário Realista projeta uma taxa de conversão conservadora e aderente às métricas de mercado para produtos digitais de segurança em modelo freemium, estimando que **2% da base total de usuários ativos cadastrados** realizem a migração para o plano *SafeID Pro*.

Considerando um crescimento orgânico moderado ao longo do primeiro ano, atingindo uma base estável de 15.000 usuários cadastrados no sexto mês, o produto alcançaria **300 assinantes ativos**.

- **Faturamento Mensal Bruto:** $300 \times R\$ 11,90 = R\$ 3.570,00$
- **Resultado Líquido:** Subtraindo os custos operacionais estáveis (reajustados proporcionalmente pelo uso de tokens e mensageria para R\$ 1.400,00), o SafeID gera um lucro líquido mensal de **R\$ 2.170,00**.

Neste cenário, o Ponto de Equilíbrio (*Break-Even Point*) da operação é atingido logo no terceiro mês, necessitando de apenas **90 assinantes ativos** para cobrir integralmente os custos fixos e variáveis básicos de infraestrutura.

5.4 Cenário Otimista

O Cenário Otimista prevê uma forte tração de mercado decorrente de picos de conscientização pública (como a repercussão na grande mídia de um megavazamento de dados no Brasil). A taxa de conversão para o plano premium é projetada em **4,5%**, impulsionada por uma estratégia de expansão *B2B2C* (parcerias corporativas onde empresas contratam o SafeID como um benefício de cibersegurança e privacidade para seus colaboradores).

Atingindo uma base de 30.000 usuários ao final do primeiro ano, com 1.350 assinantes ativos:

- **Faturamento Mensal Bruto:** $1350 \times R\$ 11,90 = R\$ 16.065,00\$$
- **Margem de Lucro Expandida:** Com o ganho de escala, o custo de infraestrutura dilui-se. Mesmo com o aumento de requisições de IA e mensageria (estimados em R\$ 2.800,00), o lucro líquido operacional salta para **R\$ 13.265,00 mensais**.

O superávit financeiro deste cenário viabiliza o reinvestimento imediato em Pesquisa e Desenvolvimento (P&D), permitindo expandir o escopo do SafeID para o desenvolvimento de extensões dedicadas à busca textual de vazamentos de chaves PIX e monitoramento de fraudes de identidade em tempo real corporativo.

5.5 Cenário Pessimista

O Cenário Pessimista antecipa um ambiente de mercado com forte resistência de adoção por parte do usuário final, alta taxa de cancelamento de assinaturas (*Churn Rate*) e uma taxa de conversão estagnada em **0,6%** (abaixo de 1%). Nesse contexto, com uma base de 10.000 usuários, a plataforma contaria com apenas 60 assinantes premium ativos.

- **Faturamento Mensal Bruto:** $60 \times R\$ 11,90 = R\$ 714,00\$$
- **Déficit Operacional:** Diante de custos fixos de manutenção da ordem de R\$ 1.055,00, o projeto operaria com um saldo negativo de **-R\$ 341,00 mensais**.

Plano de Mitigação de Riscos (Ações Contingenciais)

Para reverter o déficit do cenário pessimista e assegurar a sobrevivência do produto sem comprometer a qualidade de entrega, a equipe gestora do SafeID estabelece as seguintes ações estruturadas:

1. **Redução de Infraestrutura (*Downscaling*):** Migração temporária dos servidores lógicos para planos mínimos de computação em nuvem e limitação agressiva da retenção de dados temporários de histórico no PostgreSQL, reduzindo o custo fixo de hospedagem pela metade.
2. **Otimização de Custos de IA:** Redução do limite de tokens de saída (*Max Tokens*) do modelo e aumento da agressividade das políticas de cache do Redis, garantindo que a inteligência artificial só seja acionada em casos críticos e estritamente inéditos.
3. **Pivotagem de Canal:** Descontinuação temporária de disparos pagos de SMS, concentrando 100% das notificações e alertas de vazamentos através de e-mails criptografados e canais de disparo orgânicos de baixíssimo custo.

6 CONSIDERAÇÕES FINAIS

A conclusão deste projeto de engenharia de software consolida não apenas a entrega de uma ferramenta funcional de utilidade pública, mas também o amadurecimento técnico e analítico da equipe de desenvolvimento. O SafeID cumpre com êxito os objetivos propostos de democratizar o entendimento sobre cibersegurança e privacidade de dados, provando que a complexidade técnica de sistemas distribuídos pode ser traduzida em interfaces simples, didáticas e acessíveis ao cidadão comum.

6.1 Dificuldades, Escolhas e Descartes

O processo de desenvolvimento de um ecossistema pautado em *Security by Design* exigiu constantes avaliações de impacto entre as dimensões de custos, performance, segurança e prazos. Diante disso, o grupo deparou-se com desafios complexos que exigiram escolhas arquiteturais rígidas e o descarte consciente de soluções inicialmente planejadas.

1. Dificuldades Técnicas Encontradas

- **Gerenciamento de Taxa de Limite (*Rate Limiting*) das APIs Externas:** A principal barreira operacional residuiu na integração com a API *Have I Been Pwned*. Por tratar-se de um serviço comercial com severas restrições de requisições por segundo sob pena de bloqueio de chave, tornou-se inviável adotar uma arquitetura de chamadas síncronas diretas a partir do front-end. A solução desse impasse exigiu uma reestruturação profunda do backend, introduzindo uma camada de mensageria assíncrona baseada em filas (**BullMQ** e **Redis**) para cadenciar e suavizar o fluxo de consultas em conformidade com as diretrizes do provedor OSINT.
- **Sobrecarga Criptográfica (*Overhead*) vs. Latência:** Garantir a conformidade estrita com os artigos de privacidade da LGPD impôs o desafio de criptografar dados em repouso no banco de dados. As primeiras implementações utilizando o algoritmo **AES-256-GCM** diretamente nas rotas geraram um aumento perceptível no consumo de CPU do Node.js e na latência final da API. Para mitigar essa dificuldade, foi necessário isolar o processo criptográfico em middlewares específicos e implementar uma estratégia agressiva de cache indexado por *hashes* **SHA-256** em memória no Redis, restabelecendo o tempo de resposta do sistema para patamares abaixo de 50ms.

- **Determinismo e Custos em Inteligência Artificial:** Controlar a imprevisibilidade das respostas geradas por Grandes Modelos de Linguagem (LLMs) revelou-se um desafio complexo de *Prompt Engineering*. Inicialmente, os relatórios gerados extrapolavam os limites de tamanho (*tokens*) e, ocasionalmente, incluíam termos técnicos excessivamente herméticos. A estabilização do modelo exigiu rodadas exaustivas de parametrização de temperatura ($\$Temperature \approx 0.2\$$) e a fixação de estruturas de saída rígidas (*Structured Outputs*), garantindo um comportamento determinista, didático e financeiramente sustentável para o projeto.

2. Escolhas Arquiteturais Consolidadas

- **Adoção da *Clean Architecture*:** A escolha por isolar as regras de negócio (*Core Domain*) das ferramentas de infraestrutura (bancos de dados e APIs de IA) mostrou-se o acerto mais determinante do projeto. Essa dissociação permitiu que os testes unitários e de mutação (*Fuzz Testing*) fossem executados de forma totalmente isolada e veloz através do pipeline do **GitHub Actions**, blindando o sistema contra regressões sem a necessidade de acionar serviços externos pagos a cada build.
- **Uso Estratégico do Modelo OpenAI/Azure (6.41.0):** Em detrimento de modelos maiores e mais onerosos, a escolha pelo OpenAI/Azure (6.41.0) justificou-se pelo equilíbrio otimizado entre custo computacional, velocidade de inferência e capacidade de síntese textual. Dado que a responsabilidade do modelo limita-se a contextualizar de forma didática os metadados brutos já higienizados pelo backend, a escolha gerou uma economia de escala fundamental para a viabilidade do produto.

3. Descartes Estratégicos

- **Implementação Gradual da Infraestrutura AWS:** O planejamento arquitetural do projeto estabeleceu o ecossistema da *Amazon Web Services* (AWS) como a infraestrutura alvo (*Target Architecture*) definitiva para o ambiente de produção do SafeID, conforme detalhado na modelagem topológica do sistema (ECS Fargate, VPC isolada e RDS PostgreSQL). No entanto, para a fase de MVP (*Minimum Viable Product*) e validação do ecossistema, a equipe adotou uma estratégia de implantação incremental e financeiramente sustentável. Em vez de provisionar imediatamente os recursos corporativos de alta disponibilidade da AWS — o que geraria custos mínimos elevados em dólar e complexidade precoce de gerenciamento de redes virtuais para um ambiente de testes —, optou-se pelo empacotamento agnóstico via contêineres Docker *Multi-stage*. Essa abordagem permitiu hospedar a aplicação temporariamente em plataformas PaaS (como *Railway* e *Vercel*), garantindo agilidade no ciclo de desenvolvimento, custo

reduzido e total isolamento tecnológico. Como o código e o banco de dados foram estruturados de forma estritamente portátil, a transição para a infraestrutura nativa da AWS ocorrerá de forma fluida e gradual, acompanhando a evolução do projeto rumo ao ambiente produtivo em escala real.

- **Descarte da Fragmentação Prematura em Microsserviços:** Considerou-se inicialmente construir o SafeID em uma arquitetura puramente distribuída baseada em microsserviços (onde módulos de Inteligência Artificial, autenticação de usuários e serviços de notificação rodariam em servidores físicos isolados). Essa abordagem foi prontamente descartada porque introduziria uma complexidade de orquestração (como o uso obrigatório de Kubernetes ou múltiplos *pipelines* de CI/CD) desproporcional à necessidade atual de tráfego do sistema. O descarte dessa distribuição excessiva abriu espaço para a adoção de um **Monólito Modular** altamente coeso. Sob carga extrema, o processamento das integrações externas não afeta a renderização do sistema porque foi adotada uma solução híbrida: o servidor permanece monolítico e de fácil implantação, mas o trabalho pesado é distribuído logicamente em filas assíncronas geridas por Redis e BullMQ. Essa escolha maximizou o desempenho e reduziu drasticamente os custos operacionais do projeto.

REFERÊNCIAS BIBLIOGRÁFICAS

BROWN, T. B. et al. **Language Models are Few-Shot Learners**. arXiv preprint arXiv:2005.14165, 2020. Disponível em: <https://arxiv.org/abs/2005.14165>.

CHEN, Jing; HENRY, Elaine; JIANG, Xi. Is Cybersecurity Risk Factor Disclosure Informative? Evidence from Disclosures Following a Data Breach: J. Chen et al. **Journal of Business Ethics**, v. 187, n. 1, p. 199-224, 2023.

CHEN, Qiang; WANG, Lun. Social response and management of cybersecurity incidents. **Academic Journal of Sociology and Management**, v. 2, n. 4, p. 49-56, 2024.

CONTI, Mauro; DARGAHI, Tooska; DEGHANTANHA, Ali. Cyber threat intelligence: challenges and opportunities. **Cyber Threat Intelligence**, [s. l.], p. 1-6, 2018.

FEDERNEEL, A. **Full-Stack React, TypeScript, and Node: Build modern web applications using the Power of React, TypeScript, and Node.js**. 1. ed. [S.l.]: Packt Publishing, 2020.

FORTINET. **FortiGuard Labs report reveals 32.8 billion cyberattack attempts in Brazil in 2023**. 2024. Disponível em: <https://www.fortinet.com>. Acesso em: 18 abr. 2026.

HAVE I BEEN PWNED. **API Documentation - Version 3**. Disponível em: <https://haveibeenpwned.com/API/v3>. Acesso em: 05 mai. 2026.

IBM SECURITY. **Cost of a Data Breach Report 2023**. 2023. Disponível em: <https://www.ibm.com/security/data-breach>. Acesso em: 18 abr. 2026.

META. **React: A JavaScript library for building user interfaces**. Disponível em: <https://react.dev/>. Acesso em: 05 mai. 2026.

MICROSOFT. Azure OpenAI em Microsoft Foundry Models REST API referência. Microsoft Learn, [S. l.], 2026. Disponível em: <https://learn.microsoft.com/pt-br/azure/foundry/openai/reference>. Acesso em: 2 jun. 2026.

MOMJIAN, B. **PostgreSQL: Introduction and Concepts**. 1. ed. [S.l.]: Addison-Wesley Professional, 2001.

MOORE, G. A. **Crossing the Chasm: Marketing and Selling Disruptive Products to Mainstream Customers**. 3. ed. New York: HarperBusiness, 2014.

NEMEC ZLATOLAS, Lili; WELZER, Tatjana; LHOTSKA, Lenka. Data breaches in healthcare: security mechanisms for attack mitigation. **Cluster computing**, v. 27, n. 7, p. 8639-8654, 2024.

NODE.JS. **Node.js® is a JavaScript runtime built on Chrome's V8 JavaScript engine**. Disponível em: <https://nodejs.org/>. Acesso em: 05 mai. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL: The World's Most Advanced Open Source Relational Database**. Disponível em: <https://www.postgresql.org/>. Acesso em: 05 mai. 2026.

PRISMA. **Prisma: Next-generation ORM for Node.js & TypeScript**. Disponível em: <https://www.prisma.io/>. Acesso em: 05 mai. 2026.

REUBEN-OWOH, Blessing; HAIG, Ella. A Systematic Review of Voluntary Cybersecurity Standards and Frameworks. **International Journal of Information Security**, v. 24, n. 5, p. 206, 2025.

SANTOS, J. F.; PINTO, G. Privacidade e segurança em sistemas de informação: uma análise dos aspectos psicológicos e cognitivos envolvidos na percepção e na adoção de medidas de segurança. **Revista Interface Tecnológica**, v. 21, n. 1, p. 288–299, 2025.

SILBERSCHATZ, A.; KORTH, H. F.; SUDARSHAN, S. **Sistema de Banco de Dados**. 7. ed. Rio de Janeiro: Elsevier, 2020.

STALLINGS, W. **Criptografia e Segurança de Redes: Princípios e Práticas**. 6. ed. São Paulo: Pearson Education, 2015.

TAILWIND LABS. **Tailwind CSS: Rapidly build modern websites without ever leaving your HTML**. Disponível em: <https://tailwindcss.com/>. Acesso em: 05 mai. 2026.

TILKOV, S.; VINOSKI, S. **Node.js: Using JavaScript to Build High-Performance Network Programs**. IEEE Internet Computing, vol. 14, no. 6, p. 80-83, 2010.

VON SOLMS, Rossouw; VAN NIEKERK, Johan. From information security to cyber security. **computers & security**, v. 38, p. 97-102, 2013.

ZHANG, Xichen et al. Data breach: analysis, countermeasures and challenges. **International Journal of Information and Computer Security**, v. 19, n. 3-4, p. 402-442, 2022.